

Lecture 3 — Impact Analysis, Software Processes & Continuous Integration

Lecture id: L03 **Source decks:** ImpactAnalysis.pdf (16 slides), IntroSwProcesses.pdf (31 slides), TeamProcesses.pdf (18 slides), ContinuousIntegration.pdf (14 slides) **Labs:** AnalysisLab1.pdf (Impact Analysis Lab, 2p), CILab.pdf (Continuous Integration Lab, 1p) **Process phase(s):** Impact Analysis · Software Processes · Continuous Integration **Citation key:** Slide claims cited inline as (ImpactAnalysis p.X), (IntroSwProcesses p.X), (TeamProcesses p.X), (ContinuousIntegration p.X), (AnalysisLab p.X), (CILab p.X). Readings: [Raj13] = Rajlich, *Software Engineering: The Current Practice* (the decks explicitly cite "© 2012 Václav Rajlich ... Ch. 7 / Ch. 13"); [Bec99] = Kent Beck, *Embracing Change with Extreme Programming* (1999); [Boo] = Grady Booch, *Object-Oriented Design: With Applications*; [Tho] = ThoughtWorks, *Continuous Integration*. **Grounding note:** Every non-obvious claim below is anchored to a specific slide or lab page that I read in full (all 16 + 31 + 18 + 14 slide pages and both lab pages). Where the slides cite the Rajlich textbook chapter on a slide (e.g. "Ch. 7" on the Impact Analysis deck, "Ch. 13" on Team Processes), I record it. Numeric facts (defect densities, the agile manifesto year, the release-backlog effort numbers) are reproduced exactly as printed on the slides. Anything I add for connective tissue (e.g. tying a slide to the JHotDraw case study) is flagged as inference rather than asserted as a slide fact.

Overview

Lecture 3 sits in the middle of the eight-step change process taught in this course — **Initiation** → **Concept Location** → **Impact Analysis** → **Prefactoring** → **Actualization** → **Postfactoring** → **Conclusion** → **Verification** — and zooms in on the **Impact Analysis (IA)** phase while simultaneously stepping *up* a level to the surrounding **software processes** that schedule and repeat the whole change cycle, and *out* to the **team and tooling** infrastructure (Continuous Integration) that makes repeated change safe at scale.

The lecture answers three connected questions:

1. **Impact Analysis (deck ImpactAnalysis):** Once Concept Location has found *where* a change starts (the initial impact set), how do you systematically discover *everything else* the change touches? The answer is a graph-walking algorithm over a **class interaction graph**, propagating "change" marks (BLANK / NEXT / CHANGED / UNCHANGED / PROPAGATING / INSPECTED) until the **estimated impact set** stabilizes (ImpactAnalysis p.3–16).
2. **Software processes (decks IntroSwProcesses, TeamProcesses):** A single software change is just one **task** inside a larger **iterative process**. The lecture builds up from the **Solo Iterative Process (SIP)** for one programmer (IntroSwProcesses p.7–31), then scales to **team processes** — **AIP** (agile), **DIP** (directed), **CIP** (centralized/safeguarded), and **open source** (TeamProcesses p.3–18). It also covers process vocabulary (granularity, model vs. enactment vs. performance vs. plan), measurement (time logs, LOC, defect density), and planning (release backlog tables).
3. **Continuous Integration (deck ContinuousIntegration):** The automation backbone that lets teams "integrate their work multiple times per day," verify each integration with an **automated, self-testing build**, and keep quality high via static analysis and coverage tooling (ContinuousIntegration p.2–14).

The two labs operationalize the theory: **AnalysisLab** has students execute Rajlich's Figure 7.9 IA algorithm against a real codebase to produce an estimated impact set (AnalysisLab p.1), and **CILab** has students stand up a real GitHub Actions + Maven CI pipeline (CILab p.1).

Learning Objectives

After this lecture you should be able to:

1. **Define Impact Analysis** as "identifying the potential consequences of a change, or estimating what needs to be modified to accomplish a change" (AnalysisLab p.1), and state that it consumes the **initial impact set** from Concept Location and produces the **estimated impact set** (ImpactAnalysis p.3–4).
 2. **Build a class interaction graph** $G = (X, I)$ and compute the **neighborhood** $N(A) = \{B \mid (A, B) \in I\}$ of a class (ImpactAnalysis p.6).
 3. **Distinguish dependency from interaction**: a dependency is directional (A depends on B), an interaction is the bidirectional fact that two classes share something and so can propagate change either way (ImpactAnalysis p.5, p.9).
 4. **Execute the IA marking algorithm** with the six marks (BLANK, NEXT, CHANGED, UNCHANGED, PROPAGATING, INSPECTED) and run the propagation loop until no NEXT classes remain (ImpactAnalysis p.11–12, p.16).
 5. **Explain propagating (mailman) classes** — classes that are not themselves the target of the change but relay change between neighbors (ImpactAnalysis p.10).
 6. **Weigh change alternatives** using the two criteria *required effort* vs. *clarity of resulting code*, and recognize the short-term/long-term conflict (ImpactAnalysis p.14–15).
 7. **Place a software change inside the process granularity hierarchy** (lifecycle → stage → process → task → subtask/phase → step/action) (IntroSwProcesses p.4).
 8. **Describe SIP** and why even a solo programmer needs a defined process; read a **time log**, compute **clean time**, and read a **defect log** (IntroSwProcesses p.7–20).
 9. **Plan software changes** by analogy and decomposition, understand tasking/"epics," and read a **release backlog table** to detect on-time vs. late vs. incomplete releases (IntroSwProcesses p.21–31).
 10. **Compare team processes** AIP, DIP, CIP, and open source, including their roles (developer, tester, architect, product manager, process manager, code owner) and the Agile Manifesto's four value statements (TeamProcesses p.3–18).
 11. **State the five principles of CI**, explain "build every commit," self-testing builds, the build server as final authority, and name representative tooling categories (test frameworks, static analysis, coverage) (ContinuousIntegration p.2–14).
 12. **Set up a CI pipeline** (GitHub Actions `.yaml` + Maven) per CILab (CILab p.1).
-

Key Concepts

Impact Analysis — definition and position in the change process

What it is. Impact Analysis (IA) is the phase that "**determines the strategy and impact of change**" (ImpactAnalysis p.3) — a systematic, up-front survey of *how far a change will reach* before any code is touched. Where Concept Location answers "where does this change *start*?", IA answers "what *else* does it touch?". The deck shows the full vertical change-process diagram with **Impact Analysis** highlighted as the

phase that sits *after* Concept Location and *before* Prefactoring, with **VERIFICATION** running down the right-hand side across all phases (ImpactAnalysis p.3). Concretely:

- The **classes identified in concept location are the initial impact set** (ImpactAnalysis p.3).
- **Class dependencies are analyzed, and impacted classes are added to the impact set** (ImpactAnalysis p.3).
- The phase **produces the estimated impact set** (ImpactAnalysis p.3).

The lab gives the canonical textbook definition: change impact analysis "can be defined as 'identifying the potential consequences of a change, or estimating what needs to be modified to accomplish a change'" (AnalysisLab p.1).

What it's used for / why it matters. IA exists to *de-risk and scope* a change. Without it, a programmer edits the obvious class, declares victory, and is then surprised by ripple effects in code they never looked at — the classic source of regression faults. IA turns that surprise into a deliberate prediction: a bounded list of classes to inspect and budget for. That estimate feeds two downstream decisions — *how much effort* the change will cost (used in planning and the release backlog), and *whether the change is even worth doing here* versus somewhere else (the alternatives question below). It is the bridge between "we found the spot" and "we know the blast radius."

When & how it's applied. IA is performed once Concept Location reports its seed classes, on every non-trivial change. In practice the programmer (or a tool with the programmer in the loop) walks the class **interaction graph** outward from the seed, inspecting each reachable class and recording whether it is impacted, until the frontier stops growing — the marking algorithm detailed later. In the AnalysisLab this is run against a real Java codebase to produce a concrete table of packages and classes visited (AnalysisLab p.1).

Process anchor: IA is step 3 of the eight-step process. Its *input* is the output of Concept Location; its *output* (the estimated impact set) is what Prefactoring and Actualization will actually edit. This is why the deck's first substantive slide is the process pipeline picture (ImpactAnalysis p.3).

Initial vs. estimated impact set

What they are. These are two distinct sets, and the whole point of IA is to grow one into the other (ImpactAnalysis p.4):

- **Initial impact set** — produced by Concept Location; the seed classes where the programmer believes the change begins. It is small, often a single class, and reflects only the *visible* entry point of the change.
- **Estimated impact set** — the full set after IA repeatedly applies the "Impact Analysis" expansion outward from the initial set into the surrounding software (the slide's diagram literally shows arrows labelled "Impact Analysis" radiating from the "Initial impact set" box outward to the "Estimated impact set" region within "Software") (ImpactAnalysis p.4). The initial set is always a *subset* of the estimated set.

What they're used for. The distinction is the unit of work IA produces. The initial set is the *input* the programmer starts inspecting from; the estimated set is the *output* — the concrete to-do list of classes that Prefactoring and Actualization will edit, and the basis for the effort estimate that goes into planning. Keeping them separate is also a discipline check: if you only ever touch the initial set, you almost certainly missed ripple effects.

Why "estimated" and when it bites. It is called the **estimated** impact set because it is a *prediction* of what will change, made before the code is actually edited — it can be wrong in either direction (too large if you

over-mark cautious classes, too small if a ripple was missed) and **Verification** (the right-hand bar of the process diagram) is what later catches the gaps. Example: if Concept Location lands on `Item`, the initial set is `{Item}`; after IA walks `Item`'s neighbors the estimated set might be `{Item, Inventory, Store}` — three classes to actually budget for instead of one.

Class interactions (the relation IA walks over)

What an interaction is. An interaction is the fundamental relation IA travels along: two classes **interact if they have something in common** (ImpactAnalysis p.5). "Something in common" is what makes a change to one a *potential* change to the other — it is the channel through which impact flows. The deck gives two flavors of interaction:

- **Dependency** — "one depends on the other; there is a contract between them" (ImpactAnalysis p.5). E.g. one class calls another's method or implements its interface.
- **Coordination** — "they coordinate; they share the same coding, schedule, etc." (ImpactAnalysis p.5). E.g. two classes are wired together by a third class's logic even though neither references the other directly.

What it's used for / why it matters. The interaction relation is the *graph* IA walks; nothing else in the algorithm makes sense without it. The defining property for IA is that **interactions propagate change in both directions — from A to B or from B to A** (ImpactAnalysis p.5). This bidirectionality is *why* IA can't just follow dependency arrows one way: if you change a method's signature, the callers (who "depend on" you) must change — so impact ripples *back up* a dependency, not only down it. Treating interactions as undirected is what lets IA catch those upstream ripples.

When & how it shows up. Every edge IA marks NEXT and inspects is an interaction. In the store example, `Item`'s interactions with `Inventory` and `Price` are exactly the edges the algorithm pushes onto the frontier when `Item` is marked CHANGED. Distinguishing the two flavors matters because a coordination interaction (no direct reference) is easy to overlook when you only read import statements — yet it can still carry change.

Coordination example (ImpactAnalysis p.8): a class `C` holds an `A a` (gets a color code) and a `B b` (paints the screen); inside `foo()` the line `b.paint(a.get());` creates a **dataflow between a and b**. So `a` and `b` coordinate even though neither directly depends on the other's interface — the coordination lives in `C`'s body. (Slide attributes this to [Raj13] Ch. 7, slide 16.) Why this matters for IA: if you change the *type or meaning* of what `a.get()` returns, `b.paint(...)` is impacted even though `B` never names `A` anywhere — the only place that link is visible is the body of `C.foo()`. A dependency-only analysis would miss the `A – B` edge entirely; the interaction model captures it.

Dependency diagram vs. interaction diagram

What they are. These are two ways of drawing the *same* set of classes; they differ in which relation they record. The deck contrasts the two styles side by side (ImpactAnalysis p.9, citing [Raj13] Ch. 7 slide 17):

- **Dependency diagram** — `C` has *dashed directed arrows* down to `A` and `B` (`C` depends on `A` and `B`). Direction matters: the arrow says "C needs A," not the reverse.
- **Interaction diagram** — `C` is joined to `A` and `B` by *plain undirected lines*, and additionally `A – B` are joined directly (because they coordinate through `C`). Interactions are undirected because change flows both ways.

What each is used for, and why IA picks the interaction diagram. A dependency diagram is the right tool for questions about *build order, layering, and "what do I need to compile this"* — directional concerns. But for IA the question is "if I change one node, which others might I have to touch?", and the answer is

governed by *reachability in both directions*, plus coordination edges (the $A - B$ line) that the dependency diagram never draws at all. So IA is performed on the **interaction graph**, not the dependency graph, precisely because of bidirectional propagation and those extra coordination edges. Using the dependency diagram for IA would both lose the $A - B$ edge and wrongly let you walk arrows only one way — undercounting the estimated impact set.

Class interaction graph $G = (X, I)$ and neighborhood $N(A)$

What it is. This is the formal data structure IA operates on — a plain undirected graph (ImpactAnalysis p.6):

- $G = (X, I)$ where X = the set of classes (the nodes/vertices) and I = the set of interactions (the edges). The whole software, viewed as "classes connected by interactions."
- The **neighborhood** of a class A is $N(A) = \{ B \mid (A, B) \in I \}$ — every class directly interacting with A , i.e. one edge away.

What it's used for. Formalizing the model as a graph is what makes IA *algorithmic* rather than ad hoc: "find everything a change touches" becomes "explore the graph reachable from the seed nodes." The **neighborhood** is the specific unit the IA algorithm expands by: when a class is found to be CHANGED (or PROPAGATING), its *neighbors* $N(A)$ become the next candidates to inspect (marked NEXT). In other words, $N(A)$ defines the frontier the search grows along, and the graph being finite is what guarantees the search terminates.

When & how it's applied. The deck illustrates with a **Neighborhood of Item** UML picture: `Store` connects to `Inventory` and `Cashiers`; `Inventory` → `Item` → `Price`; `Cashiers` → `CashierRecord`. The shaded (neighbor) classes around `Item` are `Inventory` and `Price` (ImpactAnalysis p.7) — that is $N(\text{Item}) = \{\text{Inventory}, \text{Price}\}$. So if `Item` is seeded as CHANGED, exactly those two classes are pushed onto the inspection frontier first; `Store`, `Cashiers`, and `CashierRecord` are only reached if the change keeps propagating through `Inventory` or `Price`.

Propagating ("mailman") classes

What it is. A propagating class is a key subtlety of IA: a class that is **neither the origin nor a final target** of a change, but that *relays* the change between its neighbors — it carries impact through itself without itself needing new code. The deck's analogy (ImpactAnalysis p.10):

- John loaned money to Paul and now needs it back (his situation changed).
- John writes a letter to Paul; the **mailman** carries the letter from John to Paul; Paul must take a part-time job — a big change that **propagated from John to Paul**.
- John interacts with the mailman; the mailman interacts with Paul. **The change originated with John and propagates through the mailman to Paul.**

The crucial point of the analogy: the mailman's *own life* doesn't change — he just delivers the letter — yet without him the change would never have reached Paul.

What it's used for / why it matters. Propagating classes are why IA gives correct results on real code. Many classes are pure conduits — façades, delegators, container classes — that pass data through without transforming it. If IA could only mark CHANGED/UNCHANGED, such a conduit would be marked UNCHANGED (it needs no edit), the search would *stop* there, and any genuinely-impacted class on the far side would be silently missed — an under-estimated impact set and a future regression. The PROPAGATING mark solves this: it records "no code change here, but keep going."

When & how it's applied. In IA terms the mailman class is marked **PROPAGATING** — it itself is not "changed" in the sense of needing new code, but it is on the path along which change travels, so its other (BLANK) neighbors are still pushed onto the frontier and inspected. This is the difference between the basic algorithm (which only knows CHANGED/UNCHANGED) and the propagating-class algorithm. Concrete case (see Worked Example A): if `Store` merely relays changed data between `Inventory` and `Cashiers` without needing edits, marking it PROPAGATING is what forces `Cashiers` to be inspected — UNCHANGED would have wrongly halted the walk.

Status marks used during IA

What they are. The marks are a small state machine attached to each node of the interaction graph — they record *how far the inspection has progressed on that class*. Together they let the algorithm track three things at once: what's done, what's on the to-do list, and what hasn't been looked at (ImpactAnalysis p.11, [Raj13] Ch. 7 slide 19):

Mark	Meaning
BLANK	The class was never inspected and is not scheduled for inspection.
CHANGED	Programmers inspected the class and found it is impacted by the change.
UNCHANGED	Programmers inspected the class and found it is not impacted by the change.
NEXT	The class is scheduled for inspection.

Two further marks appear in the algorithm diagrams: **PROPAGATING** (the relay/mailman case, used by the "propagating classes" variant) and **INSPECTED** (used by the interactive variant in place of UNCHANGED at the loop-back point) (ImpactAnalysis p.12, p.16).

What they're used for. The marks *are* the bookkeeping that makes the IA loop correct and finite. NEXT is the work queue; BLANK is "untouched, eligible to be queued later"; CHANGED/PROPAGATING are the two outcomes that *expand* the frontier; UNCHANGED/INSPECTED are the outcome that *closes* a branch without expanding it. At the end, the estimated impact set is read off directly as "every class marked CHANGED or PROPAGATING."

When & how they're applied. Each class moves monotonically through the marks exactly once: **BLANK** → **NEXT** → {**CHANGED**, **PROPAGATING**, **UNCHANGED/INSPECTED**}. Because a class never goes backward, and the graph is finite, the loop must terminate. The marks change as you trace the algorithm: the deck's worked example (p.13) shows boxes shading from blank to hatched (NEXT) to dark (CHANGED) to a U (UNCHANGED) as the walk proceeds.

The IA algorithm (marking / propagation loop)

What it is. This is the heart of the deck and the lab: a graph-traversal procedure (a breadth-style flood-fill over the interaction graph) that mechanically grows the initial impact set into the estimated impact set by marking classes. **What it's used for:** it turns "predict the blast radius of a change" into a repeatable, checkable procedure — the same algorithm produces the same scoped class list, and the marks make it auditable. **When & how applied:** you run it once per change, starting from the Concept-Location seed, and you (or a tool) keep inspecting NEXT classes until none remain. The algorithm (ImpactAnalysis p.12 "IA including propagating classes"; p.16 "Interactive IA"; AnalysisLab Figure 7.9):

1. Create the interaction diagram and mark all classes as BLANK.

2. **Mark the class located during concept location as CHANGED** (this seeds the estimated impact set with the initial impact set).
3. **Mark all BLANK neighbors of changed classes as NEXT.**
4. **Loop:** *Are there any classes marked NEXT?* - **[No]** → stop (the estimated impact set = all classes marked CHANGED/PROPAGATING). - **[Yes]** → **Select a class marked NEXT and decide its new mark** by inspecting it:
 - **[UNCHANGED]** (or **INSPECTED** in the interactive variant) → mark it accordingly; it does not expand the frontier.
 - **[PROPAGATING]** → mark PROPAGATING, then go back and **mark its BLANK neighbors as NEXT** (the change relays onward).
 - **[CHANGED]** → mark CHANGED, then **mark its BLANK neighbors as NEXT** (the change spreads).
 - Return to the "any NEXT?" test.

The loop terminates because every class can only move BLANK → NEXT → {CHANGED, PROPAGATING, UNCHANGED/INSPECTED} once, and the graph is finite.

Two presentations of the same algorithm: - IA including propagating classes (ImpactAnalysis p.12) — the automatable/abstract version, branches **[UNCHANGED]**, **[PROPAGATING]**, **[CHANGED]**.

Interactive IA (ImpactAnalysis p.16) — a *swimlane* version split between **Computer** (creates the diagram, marks BLANK, manages the NEXT queue, applies marks) and **Programmer** (the human who selects a NEXT class and *decides* whether it is INSPECTED / PROPAGATING / CHANGED). This shows IA is a **human-in-the-loop** activity: the tooling tracks state, the programmer supplies judgement.

IA Example walk-through (ImpactAnalysis p.13): the deck shows an "Original Interaction Graph" then Concept location, then Step 1...Step 5 and Iteration 6, with boxes changing shading as they move BLANK → NEXT (hatched) → CHANGED (dark) → UNCHANGED (U). It is a visual trace of exactly the loop above over a small graph.

Alternatives and the two IA criteria

What this is. Beyond *finding* impacted classes, IA also has a *decision-making* job: when a change could be implemented in more than one place, IA **weighs the alternative places to make the change** (ImpactAnalysis p.14). The marking algorithm tells you *what* would be affected for each candidate location; this part tells you *which* candidate to pick.

- Example change request: a program displays temperature in **Fahrenheit**; change it to **Celsius**.
- There are **two separate locations** that deal with temperature: (a) where sensor data is converted to temperature, and (b) where temperature is displayed to the user.
- **The change can be done in either place**, and IA weighs these alternatives (ImpactAnalysis p.14).

What it's used for / why it matters. This is where IA stops being a pure search and becomes engineering judgement. Two implementations can both "work" yet leave the codebase in very different shapes — one cheap-but-messy, one costly-but-clean. Making the trade-off *explicit* (rather than defaulting to whatever is fastest) is how IA protects long-term maintainability. The two **criteria** for choosing (ImpactAnalysis p.15):

1. **Required effort of the change** — how much work to implement it here, now.
2. **Clarity of the resulting code** — how understandable/maintainable the code is afterward.

When & how it bites. These **often contradict each other**: it is *easier* (less effort) to just adjust the user interface, but it is *better* (clearer) to have all temperature calculations in one place. This is the **conflict between short-term and long-term goals** (ImpactAnalysis p.15) — the same tension that motivates refactoring/prefactoring and the concept of technical debt elsewhere in the course. IA's role is to surface *both* options with their costs so a human decides deliberately; it never auto-picks the low-effort option. The right call depends on context (e.g. how many more temperature-related changes you expect), which is why this stays a human-in-the-loop judgement.

Software process — why study processes at all

What a software process is. A software process is the *defined way of working* — the ordered set of activities a team or individual follows to produce and evolve software. "Study of software processes is the **core of software engineering**" (IntroSwProcesses p.1): the claim is that engineering quality comes less from any single heroic act of coding and more from the repeatable procedure around it.

What it's used for / why it matters. Studying processes is how the field accumulates and transfers know-how. You learn from **successful** past projects (the processes that worked well → distilled into a *prescription* you reuse on future projects) and from **unsuccessful** ones (diagnose the problems that led to failure so you avoid them). The slogan capturing the payoff: **Good process → Good product** (IntroSwProcesses p.1) — a reliable, well-understood process is the leading indicator of a reliable product, because it is what you can actually plan, measure, and improve.

When & how it varies. There is no single universal process; the right one depends on context. Processes **vary** along two axes (IntroSwProcesses p.2): - **Team** — organization, collaboration, skills (one person vs. a managed 50-person team needs different coordination). - **System** — technology, domain, size, expected quality (a toy script vs. avionics demand very different rigor).

This variability is exactly why the lecture presents a *family* of processes (SIP, AIP, DIP, CIP, open source) rather than one — you choose the process that fits your team and system.

Process granularity (the hierarchy that locates "a change")

What it is. Granularity is the *time/scope scale* at which you describe an activity — from "the whole life of the product" down to "inspecting one class for a minute." Processes exist at different **granularities** (IntroSwProcesses p.3–4): coarse-grained processes deal with long periods of time (*software life-span models* are the coarsest), fine-grained ones with single actions. The word "process" itself is usually reserved for things that fit within a single stage or a few neighboring stages (IntroSwProcesses p.3).

What it's used for / why it matters. The hierarchy gives the course a shared vocabulary and prevents category errors: it lets you say precisely *which level* a term belongs to, so "software change," "concept location," and "inspect a class" don't get conflated. It also explains nesting — a coarse activity is *composed of* finer ones, so improving a fine step (faster IA) measurably improves the coarse process that contains it. The full table (IntroSwProcesses p.4):

Granularity	Example
lifecycle	staged, waterfall
stage	evolution, servicing
process	SIP, AIP, DIP
task	software change, acceptance testing

Granularity	Example
subtask, phase	concept location, actualization
step, action	inspection of a class

Course anchor: This single table ties the whole course together. A **software change** is a *task*; **concept location** and **actualization** are *subtasks/phases* of that task; **inspection of a class** (exactly what IA does) is a *step/action*; and SIP/AIP/DIP are the *processes* that repeat the change task. Impact Analysis, then, is a phase composed of class-inspection steps.

Four forms of a process: model, enactment, performance, plan

What they are. The same process can be talked about in four distinct "forms," and conflating them is a classic source of confusion. They line up as *intended* → *actual* → *measured* → *predicted* (IntroSwProcesses p.5–6):

- **Process model** — the prescription of what the tasks should be and how they fit together; a **blueprint** for how to do things (IntroSwProcesses p.5). This is the *ideal/intended* form, written down in advance.
- **Enactment** — the *actual* process as it runs in the project, including **inevitable deviations and exceptions** from the model (IntroSwProcesses p.5). Reality never matches the blueprint exactly.
- **Performance** — the set of **measures** an observer of an enacted process collects (time, cost, quality, ...) (IntroSwProcesses p.6). This is the *recorded data about what actually happened*.
- **Plan** — the **expected future performance**; the decisions stakeholders take about alternative ways to enact the model (IntroSwProcesses p.6). The *forward-looking prediction*.

What they're used for / why it matters. Separating the four forms is what makes process improvement possible. You write a **model** so people know what to do; you accept that **enactment** will deviate (so you log the exceptions instead of pretending they didn't happen); you collect **performance** measures from the enactment so you have evidence, not opinion; and you turn that evidence into a **plan** for next time. The cycle is *model* → *enact* → *measure performance* → *plan* → (*refine model*).

When & how it shows up. In the SIP enactment table the "Ex — install new Bugzilla" row is exactly an *enactment* deviation the *model* did not call for; the time log that records its minutes is *performance*; the release backlog projecting effort-to-goal is a *plan*. Confusing model with enactment ("but the process says we never get interrupted") or performance with plan ("we spent 19 min, so we'll spend 19 min next time") is the trap this distinction guards against.

Solo Iterative Process (SIP)

What it is. SIP is the simplest software process: a **single programmer repeats software changes**, adding functionality **one step at a time** (IntroSwProcesses p.7) — one change request handled per iteration, then the next, looping. There is no team coordination, so it strips the process down to its essential iterative skeleton.

What it's used for / why it matters. SIP is taught first because it is the *foundation* every richer process builds on: repeated changes are the basis of software **evolution** and software **servicing / reengineering**, and SIP **demonstrates characteristics shared by all iterative processes** (IntroSwProcesses p.7) — backlogs, iterations, baselines, measurement. Learn SIP and AIP/DIP/CIP become "SIP plus team

coordination." It is also the process an individual maintainer (or a student doing the labs) actually follows in practice.

Why follow a process when solo? Even solo programmers must **meet their obligations**: fulfill promises, pay bills, plan the future, manage their own resources. SIP is the process that allows that (IntroSwProcesses p.8) — it provides the structure to commit to deadlines and track progress, *versus* just reacting flexibly to challenges with no structure (which gives no basis for planning or improvement). So "you're alone, you don't need a process" is precisely the misconception this slide rebuts.

Workproducts of SIP (IntroSwProcesses p.9): - **Product backlog** — the change requests; **represents the vision for the future of the software**; includes bugs and new demands/ideas. - **Software code**. - **Software documentation, etc.**

SIP model (IntroSwProcesses p.10): change requests flow from Users into the **Product backlog**; an **Iteration backlog** is drawn from it; the solo programmer ("Sol") performs **Software changes** → **Code update** → **Baseline** → **Iteration/release** → **Delivery** back to users, and the loop repeats.

Enactment of SIP (IntroSwProcesses p.11) — a worked task sequence using abbreviations for the change-process phases: | # | Task | Comment | |---|---|---| | 1 | **Pri** (prioritize) | New change request arrives | | 2 | **Ini** (initiation) | Add Cashier Session | | 3 | **CL** (concept location) | CashierRecord | | 4 | **IA** (impact analysis) | Estimated set has 4 classes | | 5 | **Ref** (refactoring/prefactoring) | Extract class Session | | 6 | **Ex** (exception/interruption) | Install new version of Bugzilla | | 7 | **Act** (actualization) | Replace class Session | | 8 | **Base** (baseline) | 2 regression faults added to backlog |

This table is the clearest illustration in the lecture that **Impact Analysis (task 4) is one phase among the eight**, sitting right after Concept Location and before Refactoring/Actualization, exactly as the change-process diagram claims.

Measuring SIP — time logs and clean time

Why measure at all. Measurement is the *performance* form of the process made concrete: data indicate how the process is working and serve as the foundation for future planning (IntroSwProcesses p.12). Without measurement you have opinions about how long IA "usually" takes; with it you have evidence you can plan and improve against. This is the link between enactment and plan.

Time log — what it is and what it's for. A **time log** is the raw record of where time actually went during an iteration (IntroSwProcesses p.13). It records, per step, the **Start/End** time, the number and minutes of **Interruptions**, the **Step** (Pri/Ini/CL/IA/Ref/Ex/Act/Base), and **Comments** (e.g. "IA — 4 classes," "Ref — extract Session," "Ex — downloading and installing new version of Bugzilla," "Base — 2 regression faults added to the backlog"). Its purpose is to make the *enactment* observable: you can later see which phase ate the most time, whether concept location is trending faster, and which exceptions keep recurring. How it's applied: you fill one row per step as you work, so the log doubles as a running diary of the change.

Time definitions — what they are and why both exist (IntroSwProcesses p.14): - **Total time** includes all interruptions — the wall-clock span from start to end. Useful for scheduling ("when will my desk be free?") but a poor measure of *work content*, because a 3-hour span with a 2-hour interruption is really 1 hour of work. - **Clean time** is the focused work time — interruptions removed. This is the number that *predicts future effort*, because it estimates the actual labor a phase costs independent of how your day happened to be chopped up. Analogy: in American football, **15 minutes is the clean time of each quarter** but the total elapsed time is much longer. - Formula: **Clean time = end – start – time of interruptions** (IntroSwProcesses p.14). -

How it's applied: from the worked log, IA ran 8:52–9:23 with 12 min of interruptions → clean time = 31 – 12 = **19 min**. You plan with the 19, not the 31.

Log = raw data (IntroSwProcesses p.15): the key idea here is that the time log is *not* the answer — it is the **raw material** from which answers are derived. The log can become large; from it you *aggregate* to get the useful signals: a weekly summary of clean time, the average time for concept location, whether concept location is getting faster or slower (a learning-curve signal), recurring exceptions to SIP (process problems to fix), etc. In other words: collect fine-grained data, then summarize it into the metrics that actually drive planning decisions.

Measuring program size and defects

Why size and defects. Size and defect counts are the two raw quantities that, combined, give *quality* (defects per unit size) and feed *productivity* estimates (size per unit time). The lecture's recurring caution is that these numbers are useful but blunt — treat them as order-of-magnitude, not precise.

Program size — what each measure is and what it's for (IntroSwProcesses p.16–18): - **LOC / KLOC / MLOC** — lines of source code; the **most commonly used** size measure (cheap to compute, language-neutral to count) but **very inaccurate** as a measure of *work or value* (different languages and styles give wildly different line counts for the same function). Hence the rule: **only the one or two most significant digits are meaningful** — quote *900 LOC, 23 KLOC, 3.2 MLOC*, never "23,418 LOC" as if precise (IntroSwProcesses p.16–17). Used as a rough size/normalizer (e.g. the denominator in defect density), not as a target. - **Function points** — a size measure based on delivered functionality that **correlates with LOC but is harder to compute**; intended to be more design-level than raw lines. The deck also lists measures that are *even less accurate* than LOC: number of methods, number of classes, number of files (IntroSwProcesses p.18) — convenient to count, but weak proxies for size.

Code defects — what and why (IntroSwProcesses p.19): a defect is a fault in the software — defined here as **incorrect computations** or **premature termination**. **Defect density** (defects per KLOC) is the standard *quality* metric because it normalizes raw defect counts by program size, making large and small systems comparable: - Good-quality software: **~2.0 defects per KLOC**. - Poor-quality software: higher density. - **Avionics software: ~0.1 defects per KLOC** — the cutting edge of what can be achieved (e.g. NASA Space Shuttle), shown to mark the *floor* of what's industrially attainable and at what cost.

Defect log — what it is and what it's for (IntroSwProcesses p.20): the defect equivalent of the time log. It records each defect's **found** Date/Time/Task, **Location, Description, Origin** (Date/Task where it was introduced), and **Fixed** date. Example rows: a non-terminating loop in `Cashier.get()` found during CL, origin in Act; a missing pop-up window found during Base; a `Price.get()` exception when Price = 0, origin in Ref. Its purpose is *root-cause feedback*: by linking a **symptom** (where it was found) back to the **phase that introduced it** (origin), it reveals which phase is leaking defects — so you can improve that phase rather than just patching symptoms.

Planning software changes

What planning is and why it matters. Planning is **prediction of the future** under uncertainty and risk — committing to dates and effort *before* the work is done. The lecture's central insight is that you cannot predict the future blind: **data about the past are good predictors**, so recording the past (the time/defect logs) and planning the future are closely related, and the future can't be predicted with certainty without knowing the past (IntroSwProcesses p.21). Planning is what turns measured performance into commitments stakeholders can rely on.

- **Repetitions** are the easiest things to predict, so the main lever of planning is to *maximize repetition and minimize novelty*; eliminating risk/uncertainty is a main goal of planning. Emphasize the **repetitive nature** of the process — "*repetition is the mother of skill.*" **Unique, unprecedented tasks are hard to plan** because there's no past data to lean on (IntroSwProcesses p.22). *Why it matters*: it justifies the next three bullets, which are all about manufacturing repeatability.
- **Two estimation techniques** — the concrete *how* of producing a number (IntroSwProcesses p.23): **Analogy** estimates a phase's time from similar past phases ("last coupon-style change took ~20h, so this one will too" — fast, needs a comparable precedent); **Decomposition** breaks the change into phases and sums the per-phase estimates (more work, but **errors may compensate each other**, and it works even for changes with no single close analogue). They are complementary: analogy for the whole, decomposition when no analogue exists.
- **Tasking** — *what it is/for* (IntroSwProcesses p.24): deliberately shaping changes so they are **more alike** and therefore **more predictable**; keep a **narrow range of size**, and divide large changes ("**epics**") into smaller, uniform ones. This directly enables analogy-based estimation (similar tasks → reliable past data). Example of the decomposition: "Customers can download and use sales coupons online" splits into building a web site, supporting the manager to create/remove coupons, cashing coupons at payment, and a database of coupon usage (IntroSwProcesses p.25).
- **Baselines** — *what/why/when* (IntroSwProcesses p.26): a baseline is a committed, integrated version; the planning rule is to schedule them at **regular intervals** (e.g. every day at end of shift, or every other day after a change is finished) and that **postponing is not recommended**. Frequent baselines bound how much work can be lost and keep the codebase continuously in a known-good state — the same "integrate often" logic that CI later automates.
- **Release plan** — *what/why* (IntroSwProcesses p.27): the business-facing layer of planning — committing to ship *certain functionality on a certain date*. Planning's job here is to make sure that externally-visible promise is **realistic** given the measured effort, so the business doesn't promise customers something engineering can't deliver on time.

Release backlog tables (reading effort/progress)

What it is. A **release backlog table** is a planning instrument: it lists each planned change (rows) and the **effort remaining** for each, snapshotted after x hours of work (columns) — a time series of "how much is left." **What it's for / why it matters:** it is the project's dashboard for *tracking convergence toward a release* and detecting trouble early. Reading the two summary rows over successive snapshots tells you whether you're on-time, slipping, or about to ship incomplete — *before* the deadline, while you can still react. **How it's applied:** you re-estimate remaining effort at each baseline and add a column; the trend of the bottom rows is the signal. The course shows three scenarios:

- **Original** (IntroSwProcesses p.28): 12 changes (initial, inventory, multiple prices, promo prices, cashier login, multiple cashiers, cashier sessions, detailed sale, multiple line items, payment, credit payment, check payment). **Remaining effort 330**, total effort to reach goal **330**.
- **After 100 hours** (IntroSwProcesses p.29): some estimates *grew* as work revealed more (inventory 30→50, multiple prices 30→40, cashier sessions 35→80, multiple line items 35→70). **Remaining effort 340**, total effort to reach goal **440** — i.e. 100 hours done but remaining went *up*, so the goal moved.
- **Late release** (IntroSwProcesses p.30): snapshots at 0/100/285/405/475 hours; remaining effort drives down 330 → 340 → 180 → 70 → **0**, with total effort to goal climbing to **475**. The release ships, but later than originally planned.
- **Incomplete release** (IntroSwProcesses p.31): same table but the line is drawn so that items 11 (credit payment) and 12 (check payment) are **left below the cut** — the release goes out at 405 hours *without* those two changes (remaining 70 of effort un-shipped).

Reading skill: Watch the bottom two rows. **Remaining effort** trending to 0 means you're converging; **total effort needed to reach the goal** *rising* over time means scope/estimates are inflating — that's how you spot a slipping release before the deadline.

Team Iterative Processes — overview

What & why. Team iterative processes are SIP scaled up to multiple people working concurrently. The driver is capacity: most projects need **more effort than a solo programmer can handle**, so programmers **organize into teams** (TeamProcesses p.2). **What it's used for:** the moment more than one person edits the same codebase, you inherit new problems SIP never had — parallel changes colliding, coordination overhead, who's allowed to commit — and the team processes are the different *answers* to those problems. The deck then presents three named team processes (**AIP, DIP, CIP**) plus **open source**, ordered roughly by team size and how much control/safeguarding they impose: AIP (small, consensus, no gate) → DIP (large, manager-directed, specialized roles) → CIP (DIP plus an explicit commit gate). **When to apply which:** pick by team size and risk tolerance — small trusted team → AIP; large team needing direction → DIP; large team needing tight quality control over commits → CIP.

AIP — Agile Iterative Process

What it is. AIP is an agile team process for **small-to-medium teams** where **decisions are made by consensus**, there are **no specializations** among programmers, and developers hold **only the programmer role** — everyone can do everything (TeamProcesses p.4). Its micro-cycle is **DEVELOP** → **REACT** → **MODIFY** (TeamProcesses p.4): build something, react to feedback/reality, modify accordingly, repeat.

What it's used for / why it matters. AIP suits teams small enough that consensus is fast and cheap, and that benefit from flexibility over hierarchy. No specialization means low coordination overhead and high bus-factor resilience (anyone can pick up any task), at the cost of not scaling to large teams where consensus would grind. **When & how applied:** a handful of developers pull from a shared product backlog, run parallel changes, sync at a **daily meeting**, and close each iteration at an **iteration meeting** — the consensus and lack of a commit gate distinguish it from CIP, and the lack of managers/specialists distinguishes it from DIP.

Model of AIP (TeamProcesses p.5, [Raj13] Ch. 13): requests from Users feed the **product backlog**; an **iteration backlog** is drawn; programmers run **parallel software changes**; there's a **build** and a **daily meeting**; an **iteration meeting/release** closes the loop. A **Product Manager** and a **Process Manager** are attached to the backlog/meetings.

Iterations (TeamProcesses p.6): *what it is* — the **iteration meeting** is the per-iteration planning/review checkpoint; *what it's for* — it assesses the current product state (all stakeholders participate, bringing **technical + business viewpoints** so the plan is grounded in both feasibility and value) and **plans the next iteration**, with its concrete output being an **iteration backlog extracted from the product backlog** (the slice of work the team commits to next). *When:* at the boundary between iterations — it is the seam where SIP's single-loop becomes a planned, reviewed team cadence.

Daily meeting (TeamProcesses p.7): *what it is* — a short, daily team sync; *what it's for* — the high-frequency coordination mechanism that keeps parallel work from drifting apart. It surfaces daily problems/challenges, builds consensus about progress, does **daily assignment of change requests**, clarifies ambiguities, identifies **needs for code refactoring**, and — most importantly — gives **early warning when anything goes wrong** so issues are caught in a day rather than at iteration's end. *When/how:* every day, briefly; it is

the daily counterpart to the per-iteration meeting and the human analogue of CI's "integrate often, find problems fast."

Agile Manifesto (TeamProcesses p.8): *what it is* — the founding statement of the agile movement, developed in **2001** by **17 original authors** and signed by many since; it is the value system AIP embodies. *What it's for* — it expresses *priorities*, not absolutes: each statement values the left more than the right while keeping value in both ("we value the items on the left more"). Its four value statements: 1. **Individuals and interactions** over processes and tools. 2. **Working software** over comprehensive documentation. 3. **Customer collaboration** over contract negotiation. 4. **Responding to change** over following a plan.

Why it matters / how it applies: these four lines justify AIP's design choices — consensus and the daily meeting (individuals/interactions), frequent baselines and self-testing builds (working software), stakeholder participation in iteration meetings (customer collaboration), and the DEVELOP→REACT→MODIFY loop itself (responding to change). The common exam trap is reading "X over Y" as "Y is worthless" — it means X is weighted *more*, not that documentation, contracts, or plans are abandoned.

DIP — Directed Iterative Process

What it is. DIP is a team process that runs **under the direction of managers**, with **several specialized roles** among the people involved (developer, tester, architect, product/process manager), and it **scales to large teams and large systems** (TeamProcesses p.10). Where AIP is flat and consensus-driven, DIP is hierarchical and direction-driven.

What it's used for / why it matters. Consensus and "everyone does everything" stop working past a certain team size — too many people to agree, too much code for anyone to know all of it. DIP answers both: **managers** provide direction so decisions don't require whole-team consensus, and **specialization increases effectiveness** (TeamProcesses p.12) because each person goes deep in one area (testing, architecture, etc.). The cost is coordination overhead and reduced flexibility. **When & how applied:** large projects where the product manager sets strategy, the process manager assigns tasks, developers run parallel changes, testers verify baselines, and the architect guards the design — each role visible in the DIP model diagram.

Model of DIP (TeamProcesses p.11): Users + Product manager feed the **Product backlog** → **Iteration backlog**; **Developers** run **parallel software changes**; **Testers** handle **Build**; **Process managers** run the **Iteration review/release**.

Roles — each is a *specialization* with a distinct responsibility; together they divide the labor a solo SIP programmer would do alone (TeamProcesses p.12–14): - **Developers** produce code — the people who actually implement changes. *Why separate:* lets others specialize in verification and oversight so developers can focus on building. - **Testers** verify the new baseline — independent quality checking. *Why separate:* programmers are < **50% efficient at finding their own bugs** (a figure the CI deck also cites), so a separate testing role catches what authors miss. - **Architect** — guarantees developers **preserve software architecture constraints** and **approves or disapproves commits** (TeamProcesses p.13). *What it's for:* protects the system's long-term structural integrity against erosion by many independent changes; the commit approval power makes the architect a quality gate (and is the seed of CIP's explicit "permission to commit"). - **Product managers** make **strategic** decisions (what to build, business priorities); **Process managers** assign tasks and control the process (how the work flows) (TeamProcesses p.14). *Why two managers:* it separates *what/why* (product) from *how/when* (process). - There can be **additional specialized roles**, because **specialization increases effectiveness** (TeamProcesses p.12) — deeper expertise per person, at the price of more coordination between roles.

CIP — Centralized (Safeguarded) Iterative Process

What it is. CIP (TeamProcesses p.15–16) is DIP plus a **commit safeguard**: a checkpoint that no change reaches the shared codebase without explicit approval. In the model, Users + Product Manager feed the **product backlog**; Developers run **parallel software changes** but need **permission to commit** granted by **Architects and Code owners**; Testers handle **build**; **Process Manager** runs **release**.

What it's used for / why it matters. The added "**permission to commit**" gate exists to protect quality and architectural integrity when the codebase is large, valuable, or worked on by developers of varying trust/skill — you cannot afford a bad commit to slip straight into mainline. By routing every commit through architects/code owners (people who own that part of the system), CIP catches design violations and risky changes *before* integration rather than after. The trade-off is reduced developer autonomy and a potential bottleneck at the gate. **When applied:** large or safety-/quality-critical projects, and notably **open source** (next), where contributors are a wide, variable-skill community and a gate is essential. The distinguishing feature versus DIP is precisely this explicit gate controlled by architects/code owners — DIP's architect *can* approve/disapprove commits, but in CIP that approval is a *mandatory, centralized step*.

Code ownership and open source

- **Code ownership** — *what it is*: an organizing scheme where programmers **specialize in certain parts of the code**, each "owning" a subsystem (TeamProcesses p.17). *What it's for*: deep familiarity and accountability — the owner knows their area thoroughly and approves changes to it (which is how CIP's gate is staffed). *The downside / when it bites*: **coordination can become a problem** — a change spanning several owners' areas needs all of them, and an owner becomes a bottleneck or single point of failure. This is the **direct contrast** with CI's principle that the **team owns the code, not the individual** (ContinuousIntegration p.5): CI deliberately rejects exclusive ownership to keep anyone from blocking integration, trading deep ownership for fluid collective change.
- **Open Source Development** — *what it is*: the development model of open communities, characterized as **safeguarded** (a commit gate, like CIP), using **code ownership**, with a **wide community of developers of variable skills** (TeamProcesses p.18). *Why those features*: because contributors are numerous, anonymous, and of unknown skill, the project *must* safeguard the mainline (gate every contribution) and lean on trusted code owners/maintainers to review — exactly the conditions CIP was built for. It is effectively CIP applied to an open, untrusted contributor pool.

Continuous Integration — definition and value

What it is. CI = **Continuous Update for the whole software process** — the practice (and the tooling backbone) of merging everyone's work frequently and verifying each merge automatically. Concretely, teams **integrate their work multiple times per day**, and **each integration is verified by an automated build** (ContinuousIntegration p.2). It is the automated, machine-enforced version of the "integrate often / baseline often" discipline the process slides preached manually.

What it's used for / why it matters. CI exists to kill **integration hell** — the painful, defect-laden big-bang merge that happens when developers work in isolation for weeks and only combine at the end. By integrating constantly, conflicts and breakages are small and caught within hours, so CI **significantly reduces integration problems** and lets you **develop cohesive software more rapidly** (ContinuousIntegration p.2). It is what makes *frequent change safe at team scale* — the enabling infrastructure behind AIP/DIP/CIP's parallel changes.

When & how applied. The deck's cycle image shows the mechanism: **Commit → (Initiate CI Process) → Build → Test → Report → (back to) Development**, all turning around **Source Control**

(ContinuousIntegration p.2). Every commit to the shared repository automatically triggers a build-and-test run, and the result is reported back to developers immediately — that feedback loop, running many times a day, is CI in action (realized in the CILab as GitHub Actions + Maven).

The CILab adds historical grounding: CI is the practice of **merging all developers' working copies to a shared mainline several times a day** [Tho]; **Grady Booch first proposed the term CI in his 1991 method** (though he did not advocate integrating several times a day) [Boo]; **Extreme Programming (XP) adopted CI and advocated integrating more than once per day — perhaps tens of times per day** [Bec99] (CILab p.1).

The five principles of CI

What this is: the five rules that, together, make CI work — each addresses one way integration can go wrong (ContinuousIntegration p.3): 1. **Environments based on stability** — *promote code through progressively stricter environments so quality is proven before production.* 2. **Maintain a code repository** — *one shared source of record everyone integrates into; nothing exists "only on a laptop."* 3. **Commit frequently and build every commit** — *small, frequent integrations so problems are small and caught immediately.* 4. **Make the build self-testing** — *the build proves its own correctness by running tests, so a green build means working software, not just compiling software.* 5. **Keep the build fast** — *fast feedback; a slow build defeats "build every commit" because developers stop waiting for it.*

Why they hang together: principle 3 (commit every change) is only safe if the build is self-testing (4) and fast (5), all of which require a single repository (2) and a promotion pipeline (1). Drop any one and the others weaken — e.g. a self-testing build that takes an hour (violating 5) discourages frequent commits (violating 3).

CI principle details

- **Environments based on stability** (ContinuousIntegration p.4): *what* — create server environments that **model code stability** and **promote code to stricter environments as quality improves**, the classic **Dev** → **Test** → **Stage** → **Prod** promotion pipeline. *Why* — it provides a gradient of risk: code is proven safe in cheap, forgiving environments before it ever reaches the expensive, unforgiving production one, so failures surface where they're harmless. *How* — a change only advances to the next stage after passing the current stage's checks.
- **Commit frequently / build every commit** (ContinuousIntegration p.5): *what* — **change your habits**: commit **small, functional changes**, write **unit tests**, and remember the **team owns the code, not the individual**. *Why* — small commits keep each integration's blast radius tiny (easy to diagnose and revert), unit tests make the build self-verifying, and collective ownership stops anyone from gatekeeping or hoarding code. *How* — this is the developer-side behavior change CI demands, contrasting directly with the code-ownership model of CIP/open source.
- **"The code builds on my box..."** (ContinuousIntegration p.6): *what* — the **source-code repository is the source of record** and the **build server settles disputes: it only gets code from the repo** and is the **final authority on stability/quality**. *Why it matters* — it eliminates the "works on my machine" excuse by making one neutral, reproducible environment the arbiter; if it doesn't build from the repo, it doesn't build, full stop. *When* — every disagreement about whether code "works" is resolved by what the build server does, not what any laptop does.
- **Why build every commit** (ContinuousIntegration p.7): *what/why* — justified by the agile principle **"if it hurts, do it more often"**: difficult activities (like integration) get easier the more frequently you do them, and frequent builds **reduce the time between defect introduction and removal** so bugs are found while the cause is still fresh. *How* — therefore **automate the build** so doing it on every commit costs nothing human. The justification is mechanical (shrink the defect-detection gap), not ceremonial.

Testing inside the CI build

- **Add testing to build** (ContinuousIntegration p.8): *what/why* — the rationale for making the build self-testing rests on a defect-detection fact: individual programmers are < **50% efficient at finding their own bugs** (they're blind to their own mistakes), and **multiple quality methods discover more defects** — combining **3 or more methods yields > 90% defect removal**. The **most effective methods** are **design inspections, code inspections, and testing**. *How it applies*: this is why CI layers automated testing on top of human inspection rather than relying on either alone — no single method, especially the author's own checking, is enough.
- **Self-testing builds** (ContinuousIntegration p.9): *what* — a build that runs its own tests to prove the code works. The deck distinguishes two test kinds by purpose: **System tests** are end-to-end and often take minutes-to-hours (thorough but slow), while **unit tests** are **fast** (no database or file system), **focused** (pinpoint exactly which unit broke), and are therefore the **best method for verifying builds**. *Why unit tests for CI*: their speed serves "keep the build fast" and their focus makes failures instantly diagnosable — both essential when you build on every commit.

Automated quality, build-server hardware, tooling

- **Automated quality with CI** (ContinuousIntegration p.10): *what/why* — beyond pass/fail tests, CI runs automated quality analysis that catches defects and decay tests can't. **Static code analysis** reads the source *without running it* to flag common Java bugs (**Findbugs, PMD**) and check code-style compliance (**Checkstyle**); **unit-test analysis** measures **coverage (Cobertura)** — how much code the tests actually exercise — and finds **hotspots** of low testing + high complexity (**SONAR**), i.e. the riskiest, least-tested code. *How it's used*: these run automatically on each build so quality erosion is reported continuously, not discovered at release.
- **Build server hardware** (ContinuousIntegration p.11): *what/why* — these analyses aren't free: Maven + Java demand lots of **memory**; compile + unit test demand lots of **CPU**; static analysis demands **lots and lots** of CPU. *Why it's called out*: because heavy analysis directly threatens principle 5, hence the recurring imperative "**Please, KEEP IT FAST.**" *How*: provision the build server generously and keep the pipeline lean so the feedback loop stays short enough to run on every commit.
- **Test frameworks** (ContinuousIntegration p.12): *what they are* — libraries that define, run, and report tests in the self-testing build: **JUnit, NUnit, MSTest** (unit testing), **Selenium** (browser/UI end-to-end), **FitNesse** (acceptance testing). *Used for*: writing the automated tests principle 4 depends on.
- **Static analysis tools** (ContinuousIntegration p.13): *what they are* — tools that inspect code without executing it, for bugs, style, duplication, and security: **Checkstyle, CodeScanner, DRY, Crap4j, Findbugs, PMD, Fortify, Sonar, FXCop**. *Used for*: automated code-quality and compliance gates in the build.
- **Code coverage tools** (ContinuousIntegration p.14): *what they are* — tools that measure which lines/branches the tests actually run: **Emma, Cobertura, Clover, GCC/GCOV**. *Used for*: revealing untested code so coverage gaps are visible. The slide shows a coverage view (green = covered lines, red = uncovered, "1 of 2 branches missed") over an `addAll(int index, Collection c)` method — exactly the kind of "this branch is never tested" gap coverage tools exist to surface.

The change-process phase abbreviations (Pri / Ini / CL / IA / Ref / Ex / Act / Base)

What they are. The SIP enactment table (IntroSwProcesses p.11) and the time log (IntroSwProcesses p.13) use a fixed set of two-to-four-letter abbreviations for the steps a programmer records. Knowing the expansion of each is required to read any log row the exam shows you:

Abb rev.	Expansion	Example from the slides
Pri	Prioritize (backlog management)	"New change request arrives" — logged 8:23–8:31 (IntroSwProcesses p.11, p.13)
Ini	Initiation	"Add Cashier Session" — logged 8:32–8:39 with 1 interruption of 2 min (p.11, p.13)
CL	Concept Location	"CashierRecord" — logged 8:42–8:52 (p.11, p.13)
IA	Impact Analysis	"Estimated set has 4 classes" — logged 8:52–9:23 with 2 interruptions totalling 12 min (p.11, p.13)
Ref	Refactoring (here prefactoring, since it precedes Act)	"Extract class Session" — logged 9:27–10:46 with 3 interruptions totalling 25 min (p.11, p.13)
Ex	Exception (deviation from the process model)	"Install new version of Bugzilla" / "Downloading and installing new version of Bugzilla" — logged 10:50–11:42 with 2 interruptions totalling 6 min (p.11, p.13)
Act	Actualization	"Replace class Session" — logged 1:23–2:17 (p.11, p.13)
Base	Baseline	"2 regression faults added to backlog" — logged 2:22–3:12 with 3 interruptions totalling 12 min (p.11, p.13)

Why it matters / two traps. First trap: **Pri** and **Ex** are *not* phases of the eight-step change process. The change-process diagram (ImpactAnalysis p.3) lists Initiation → Concept Location → Impact Analysis → Prefactoring → Actualization → Postfactoring → Conclusion with Verification cross-cutting; *Pri* is backlog prioritization that happens *before* a change is initiated, and *Ex* is an **enactment deviation** — exactly the "inevitable deviations and exceptions from the process model" that distinguish enactment from model (IntroSwProcesses p.5). Second trap: in this enactment **Ref** appears *before* **Act**, so it plays the role of *prefactoring* (preparing the code — "Extract class Session") rather than postfactoring; the actualization then *replaces* class Session (IntroSwProcesses p.11).

Verification as the cross-cutting right-hand bar

What it is. On the change-process diagram the seven phases are stacked vertically (Initiation, Concept Location, Impact Analysis, Prefactoring, Actualization, Postfactoring, Conclusion) while **V-E-R-I-F-I-C-A-T-I-O-N** runs letter-by-letter down the entire right-hand side (ImpactAnalysis p.3). The drawing is the claim: verification is **not a final phase** appended at the end — it spans *every* phase of the change.

Why it matters for IA specifically. IA's output is the **estimated** impact set — a prediction made before editing (ImpactAnalysis p.3–4). Predictions can be wrong, and the cross-cutting verification bar is what catches mis-estimates: in the SIP enactment, the **Base** step finds **2 regression faults** that are added to the backlog (IntroSwProcesses p.11, p.13) — that is verification-at-baseline detecting consequences the earlier phases (including IA) missed. Reading the diagram correctly ("verification happens throughout") versus incorrectly ("verification is step 8") is a standard exam discrimination.

Static vs. dynamic analysis in the IA lab objective

What the lab asks. The AnalysisLab objective is to "**apply static and dynamic analysis** to find the estimated impacted set of classes based on your Change request" (AnalysisLab p.1) — i.e. the lab explicitly names *two* analysis lenses, not one.

What each lens is (in this lecture's terms). *Static analysis* inspects the source without running it — it is how you build the interaction diagram from code (declarations, calls, contracts), and it is the same idea the CI deck automates with static code analysis tools (Findbugs, PMD, Checkstyle) that "look for common java bugs"

without execution (ContinuousIntegration p.10, p.13). *Dynamic analysis* observes the **running** program. **Why the lab wants both:** purely static reading is good at finding *dependency*-type interactions ("one depends on the other; there is a contract between them," ImpactAnalysis p.5) but can miss *coordination*-type interactions, whose evidence is dataflow at runtime — the deck's own example is `b.paint(a.get());` creating a "dataflow between a and b" inside a third class `C`, an edge invisible if you only scan `A`'s and `B`'s declarations (ImpactAnalysis p.8). Executing the feature and watching which classes participate (dynamic analysis) surfaces exactly those edges, so the estimated impact set is not undercounted.

The INSPECTED-vs-UNCHANGED labeling subtlety across the two algorithm diagrams

What the slides literally show. The two activity diagrams are the *same* algorithm but label the "not impacted" outcome differently, and the exam loves this nuance:

- On **"IA including propagating classes"** (ImpactAnalysis p.12) the *decision branch* is guarded **[UNCHANGED]**, but the *action box* that branch leads to reads **"Mark class as INSPECTED."** So even the non-interactive diagram uses INSPECTED as the recorded mark for a class whose inspection verdict was "unchanged."
- On **"Interactive IA"** (ImpactAnalysis p.16) the branch guard itself is **[INSPECTED]** — verdict and mark use the same word — and the activity is split across **Computer** and **Programmer** swimlanes.

How to keep it straight. UNCHANGED is the *inspection verdict* ("the programmers inspected the class and found that it is not impacted by the change," ImpactAnalysis p.11); INSPECTED is the *bookkeeping mark* the algorithm records for that verdict in the activity diagrams (ImpactAnalysis p.12, p.16). Both close the branch without expanding the frontier; neither adds the class to the estimated impact set. If an exam question lists the marks, the safe complete answer is: BLANK, NEXT, CHANGED, UNCHANGED (verdict table, p.11), plus PROPAGATING and INSPECTED (algorithm diagrams, p.12 and p.16).

Mapping SIP artifacts onto the four process forms

The mapping. The four "forms of process" (IntroSwProcesses p.5–6) are abstract until you pin each to a concrete SIP artifact from the same deck — then the whole measurement-and-planning story snaps into one picture:

Process form	What it is (slide wording)	The SIP artifact that embodies it
Process model	"prescription what the tasks should be and how should they fit together; a blueprint how to do things" (IntroSwProcesses p.5)	The SIP model diagram — Users → change requests → product backlog → iteration backlog → software changes → code update → baseline → iteration/release → delivery (IntroSwProcesses p.10)
Enactment	"the actual process in the project; inevitable deviations and exceptions from the process model" (p.5)	The Enactment of SIP table — Pri/Ini/CL/IA/Ref/ Ex /Act/Base, where the Ex row (install new Bugzilla) is precisely a deviation the model never prescribed (p.11)
Performance	"set of measures that an observer of an enacted process collects: time, cost, quality, ..." (p.6)	The time log (p.13) and the defect log (p.20) — raw measured data about the enactment
Plan	"expected future performance; decisions that the project stakeholders take; alternatives how to enact the process model" (p.6)	The release backlog tables — per-change remaining-effort estimates projected forward (p.28–31)

Why it matters. Exam questions often hand you one artifact (a time log, a backlog table) and ask which *form* it instantiates. The mapping also explains the improvement loop: the model is enacted, the enactment is measured (performance), the measurements feed the plan, and the plan informs how the model is enacted next time (IntroSwProcesses p.5–6, p.12, p.21).

JHotDraw Connection

Direct slide mentions: none. None of the four decks or two labs in *this* Lecture-3 folder names JHotDraw on any slide — the running example used throughout these particular slides is the **cash-register / store domain** (`Store`, `Inventory`, `Item`, `Price`, `Cashiers`, `CashierRecord`, `Cashier.get()`, `Session`, `cashier` sessions, `payment`) (ImpactAnalysis p.7, p.8; IntroSwProcesses p.10–11, p.13, p.20, p.28–31).

How it ties to JHotDraw (course-level inference, not a slide fact): JHotDraw is the standard case study for the *applied* portions of SB5-MAI, and the **AnalysisLab is where the IA algorithm is run against a real Java codebase** — students "find the estimated impact set of classes" and tabulate **packages and number of classes visited** after concept location (AnalysisLab p.1–2). In this course that target codebase is JHotDraw, so the IA marking algorithm (BLANK → NEXT → CHANGED/PROPAGATING/UNCHANGED) is what you would execute over JHotDraw's class interaction graph to scope a drawing-tool change. Likewise the **CILab** has you stand up GitHub Actions + Maven CI for *your project* (CILab p.1), which in the lab sequence is the JHotDraw-based repository. Treat the cash-register names on the slides as pedagogical stand-ins; the lab transfers the same mechanics to JHotDraw.

Worked Example / Process Walkthrough

Worked example A — Running the IA algorithm on the Item neighborhood

Take the store interaction graph from the deck (ImpactAnalysis p.7): edges are `Store-Inventory`, `Store-Cashiers`, `Inventory-Item`, `Item-Price`, `Cashiers-CashierRecord`. Suppose Concept Location landed on `Item` (the initial impact set = {`Item`}). Execute the algorithm (ImpactAnalysis p.12/p.16):

1. **Mark all BLANK:** `Store`, `Inventory`, `Item`, `Price`, `Cashiers`, `CashierRecord` = BLANK.
2. **Seed:** mark `Item` = **CHANGED** (from concept location).
3. **Expand frontier:** `Item`'s BLANK neighbors are `Inventory` and `Price` → mark both **NEXT**.
4. **Any NEXT? Yes.** Select `Inventory`, inspect: - Say the change to `Item`'s interface forces `Inventory` to adapt → **CHANGED**. Its BLANK neighbor `Store` → **NEXT**.
5. **Any NEXT? Yes.** Select `Price`, inspect: - Say `Price` merely *reads* an unchanged `Item` field → **UNCHANGED**. No new frontier.
6. **Any NEXT? Yes.** Select `Store`, inspect: - Say `Store` only holds references and relays calls between `Inventory` and `Cashiers` without itself needing edits, but it *does* pass changed data through → **PROPAGATING**. Its BLANK neighbor `Cashiers` → **NEXT**.
7. **Any NEXT? Yes.** Select `Cashiers`, inspect → **UNCHANGED** (the change never reaches the cashier side). No new frontier. `CashierRecord` is never even reached (stays BLANK).
8. **Any NEXT? No** → **stop**.

Estimated impact set = classes marked CHANGED or PROPAGATING = {`Item`, `Inventory`, `Store`}. `Price`, `Cashiers` are UNCHANGED; `CashierRecord` was never inspected (BLANK). Note how `Store` being

PROPAGATING (the mailman) is what forced **Cashiers** to be inspected even though Store itself needed no code change — that's the value of the propagating-class refinement (ImpactAnalysis p.10).

Worked example B — The Fahrenheit→Celsius alternative decision

Change request: display Celsius instead of Fahrenheit (ImpactAnalysis p.14). IA finds **two candidate locations**: (a) sensor-to-temperature conversion, (b) temperature-to-display. Apply the two criteria (ImpactAnalysis p.15):

- **Option (b) — change the UI only**: low **required effort**, but it scatters temperature logic; later changes will need to touch multiple display sites. Good short-term, poor long-term **clarity**.
- **Option (a) — convert centrally**: higher effort now, but **all temperature calculations live in one place** → better **clarity** and cheaper future changes.

This is the **short-term vs. long-term conflict**. The "right" answer depends on whether more temperature changes are expected; IA's job is to surface both options *with their costs* so the programmer can decide — it does not mechanically pick one.

Process walkthrough — one SIP iteration end-to-end

Following the SIP enactment table (IntroSwProcesses p.11) plus the time log (p.13), a single change to "Add Cashier Session" runs:

Pri (a new change request arrives) → **Ini** "Add Cashier Session" → **CL** locates concept at **CashierRecord** (8:42–8:52) → **IA estimates 4 classes** (8:52–9:23, interrupted twice for 12 min) → **Ref** extract class **Session** (9:27–10:46) → **Ex** download/install new Bugzilla (an interruption-style exception) → **Act** replace class **Session** (1:23–2:17) → **Base** baseline, during which **2 regression faults are found and added to the backlog** (2:22–3:12).

Clean time for IA = 9:23 – 8:52 – 12 min interruption = 31 – 12 = **19 min** (using *Clean time = end – start – interruptions*, IntroSwProcesses p.14). Those two baseline regression faults flow into the **defect log** (p.20) and the **product backlog** (p.9), feeding the *next* iteration — closing the SIP loop (p.10).

CILab walkthrough — minimal pipeline

Per CILab (CILab p.1): (1) follow GitHub's "Building and testing Java with Maven"; (2) add a ***.yaml** under **<YOUR_PROJECT>/github/workflows/** to instruct GitHub Actions; (3) configure it to **build on every pull request** with Maven; (4) add a **.maven-settings.xml** in the project root to consume **shared jars from GitHub Packages**; (5) configure the **.yaml** to **run tests automatically**. The result realizes the deck's principles "build every commit" and "make the build self-testing" (ContinuousIntegration p.3, p.5, p.9).

Worked example C — Time-log arithmetic for every row

The full time log (IntroSwProcesses p.13) has eight rows; the columns are **Start, End, # of interruptions, interruption minutes, Step, Comments**. Applying *Clean time = end – start – time of interruptions* (IntroSwProcesses p.14) to every row:

Step	Start	End	Interruptions (count)	Interruption min	Total time (min)	Clean time (min)	Comment on the slide
Pri	8:23	8:31	—	—	8	8	(new change request prioritized)
Ini	8:32	8:39	1	2	7	5	Add Cashier Session

Step	Start	End	Interruptions (count)	Interruption min	Total time (min)	Clean time (min)	Comment on the slide
CL	8:42	8:52	—	—	10	10	CashierRecord
IA	8:52	9:23	2	12	31	19	4 classes
Ref	9:27	10:46	3	25	79	54	extract Session
Ex	10:50	11:42	2	6	52	46	Downloading and installing new version of Bugzilla
Act	1:23	2:17	—	—	54	54	class Session
Base	2:22	3:12	3	12	50	38	2 regression faults added to the backlog

Observations worth being able to reproduce:

- Rows with no interruption entries (Pri, CL, Act) have **clean time = total time** — the formula's interruption term is simply 0 (IntroSwProcesses p.13–14).
- The **gaps between rows** (8:39→8:42, 9:23→9:27, 10:46→10:50, 11:42→1:23) are time *between* steps — including the long midday break before Act — and belong to **no** step; total time per step is measured within the row, not across rows (IntroSwProcesses p.13).
- The slide labels arrows pointing at the log's parts: the Step/Comments column sequence is the **process enactment**, the count/minutes columns are the **interruption** record, and the first two columns are **Start** and **End** (IntroSwProcesses p.13) — i.e. one physical table captures both the *enactment* (which steps ran, in what order, including the Ex deviation) and the *performance* (the minutes).
- The most interrupted step here is Ref (3 interruptions, 25 min) — its clean 54 min vs. total 79 min is the deck's own best illustration of why planning from total time would systematically overestimate effort (IntroSwProcesses p.13–14).

Worked example D — Defect log rows traced end-to-end

The defect log (IntroSwProcesses p.20) has columns **Defect # / Found (Date, Time, Task) / Location / Description / Origin (Date, Task) / Fixed**. The three example rows in full, each with its lesson:

1. **Defect 1** — found **11/4 at 9:00 during CL**, location `Cashier.get()`, description "**for I = 0, loop does not terminate**"; origin **3/12, Act**; fixed **12/8**. *Lesson*: a defect introduced during an **Actualization months earlier** (3/12) was discovered not by testing but by a programmer **reading code during Concept Location** (11/4) — code reading is a defect-detection activity, and the origin column is what lets you attribute the leak to the Act phase (IntroSwProcesses p.20).
2. **Defect 2** — found **11/4 at 2:32 during Base**, location `--` (not localized), description "**The pop-up window for 3rd cashier does not appear**"; origin `?` (unknown); fixed **11/25**. *Lesson*: the log records *what is known when it is known* — a defect can be observed as a symptom (missing pop-up) before anyone knows which class is at fault or which task introduced it; the `--` and `?` entries are legitimate states, not omissions (IntroSwProcesses p.20).
3. **Defect 3** — found **11/4 at 3:02 during Base**, location `Price.get()`, description "**Price = 0 raises exceptions**"; origin **4/21, Ref**; fixed-date cell left blank on the slide (still open). *Lesson*: the origin task is **Ref** — i.e. a **refactoring introduced a defect**. Refactoring is intended to be behavior-preserving, but the log shows it is not risk-free, which is part of why baselines and verification run after every change (IntroSwProcesses p.20, p.26; ImpactAnalysis p.3).

Reading skill: the **Found Task** column tells you *which phase detects* defects (here: CL once, Base twice — baselining is a major detection point, consistent with "2 regression faults added to backlog" in the enactment, IntroSwProcesses p.11), while the **Origin Task** column tells you *which phase leaks* defects (here: Act and Ref). Process improvement targets the origin phases; detection capacity lives in the found phases.

Worked example E — Reading all three release-backlog scenarios column by column

All three scenario tables (IntroSwProcesses p.29–31) share the same 12 rows; the original plan is the 0-hours column. Numbers in full:

Initial estimates (the "0" column, also the "original" table of p.28): 1 initial = 10; 2 inventory = 30; 3 multiple prices = 30; 4 promo prices = 30; 5 cashier login = 20; 6 multiple cashiers = 30; 7 cashier sessions = 35; 8 detailed sale = 30; 9 multiple line items = 35; 10 payment = 20; 11 credit payment = 40; 12 check payment = 20. **Remaining effort = 330; total effort needed to reach the goal = 330** (IntroSwProcesses p.28–29).

After 100 hours (p.29): four estimates grew — inventory 30→**50** (+20), multiple prices 30→**40** (+10), cashier sessions 35→**80** (+45, the worst surprise), multiple line items 35→**70** (+35); the other eight rows are unchanged. **Remaining effort = 340; total effort needed to reach the goal = 440.** The key arithmetic identity: *total effort to reach the goal = hours already worked + remaining effort* → 100 + 340 = 440. So although 100 hours were spent, the remaining work went *up* by 10 — the goal moved 110 hours further away than the original 330 (IntroSwProcesses p.29).

Late release (p.30): snapshots at **0 / 100 / 285 / 405 / 475** hours. Additional estimate growth after the 100-hour column: multiple cashiers 30→**55** (+25, visible from the 285 column) and credit payment 40→**50** (+10, visible from the 405 column). The two summary rows over the five snapshots: **remaining effort 330 → 340 → 180 → 70 → 0** and **total effort needed to reach the goal 330 → 440 → 465 → 475 → 475**. Verify the identity at each snapshot: 285+180 = 465 ✓; 405+70 = 475 ✓; 475+0 = 475 ✓. The release does complete (remaining hits 0) but at **475 hours against an original 330-hour plan** — about 145 hours (~44%) over (IntroSwProcesses p.30).

Incomplete release (p.31): identical numbers through the 405-hour snapshot (remaining 330 → 340 → 180 → 70; total-to-goal 330 → 440 → 465 → 475) — but there is **no 475 column**: the release ships at 405 hours with **70 hours of estimated work un-shipped**. That 70 is exactly items **11: credit payment (50)** and **12: check payment (20)** — 50 + 20 = 70 ✓ — the two changes cut from the release (IntroSwProcesses p.31).

How to answer "which scenario is this?": remaining effort reaches 0 on the planned budget → on-time; remaining effort reaches 0 but total-to-goal exceeded the original total → **late release**; the table simply stops with remaining effort > 0 and the bottom items dropped → **incomplete release** (IntroSwProcesses p.29–31).

Worked example F — Mapping the CILab steps onto the five CI principles

Each numbered classwork step of the CILab (CILab p.1) realizes specific principles from the deck (ContinuousIntegration p.3):

CILab step (CILab p.1)	What you do	CI principle(s) realized
1. Go to "Building and testing Java with Maven"	Adopt the documented GitHub-Actions-plus-Maven build recipe	Automate the build (ContinuousIntegration p.7) — the precondition for building every commit

CILab step (CILab p.1)	What you do	CI principle(s) realized
2. Add a <code>*.yaml</code> to <code><YOUR_PROJECT>/github/workflows/</code>	Tell GitHub Actions CI what to do	Maintain a code repository as the single source of record — the pipeline definition itself lives in the repo (ContinuousIntegration p.3, p.6)
3. Configure the <code>*.yaml</code> to automatically build for each pull request (use Maven)	Every proposed integration triggers a build	Commit frequently and build every commit (ContinuousIntegration p.3, p.5, p.7)
4. Create <code>.maven-settings.xml</code> in the project root to use shared jars from GitHub Packages ("Working with the Apache Maven registry")	Resolve shared team artifacts from a registry, not from someone's laptop	Build server only gets code (and dependencies) from the repo/registry — kills "it builds on my box" (ContinuousIntegration p.6)
5. Configure the <code>*.yaml</code> to execute tests automatically	The build proves itself with tests	Make the build self-testing (ContinuousIntegration p.3, p.9)

The remaining principles are properties you must *preserve* while doing the steps: **environments based on stability** (promotion to stricter environments as quality improves, ContinuousIntegration p.4) and **keep the build fast** (unit-test-heavy verification, adequate build-server resources, ContinuousIntegration p.9, p.11).

Definitions & Terminology

Each definition states **what** the term is, **what it's for**, and (where useful) **when/how** it applies.

Term	Definition	Source
Impact Analysis (IA)	<i>What:</i> identifying the potential consequences of a change, or estimating what needs to be modified to accomplish it; determines the strategy and impact of change. <i>For:</i> scoping and de-risking a change before editing, by predicting its blast radius. <i>When:</i> step 3 of the change process, after Concept Location, run by walking the interaction graph.	AnalysisLab p.1; ImpactAnalysis p.3
Initial impact set	<i>What:</i> the classes identified during Concept Location — the seed for IA. <i>For:</i> the starting point IA expands from; always a subset of the estimated set. <i>When:</i> it is IA's <i>input</i> , handed over from Concept Location.	ImpactAnalysis p.3–4
Estimated impact set	<i>What:</i> the full set of classes IA predicts will be affected — IA's output. <i>For:</i> the concrete to-do list Prefactoring/Actualization edit, and the basis of the effort estimate. <i>When:</i> read off at the end of the marking loop as all CHANGED+PROPAGATING classes; "estimated" because it's a prediction Verification later checks.	ImpactAnalysis p.3–4
Interaction	<i>What:</i> two classes have something in common (dependency or coordination) so change can propagate either direction between them. <i>For:</i> it is the edge IA walks along — the channel impact flows through. <i>When:</i> every NEXT class is reached across one interaction edge.	ImpactAnalysis p.5
Dependency	<i>What:</i> a directional relation — one class depends on another via a contract (e.g. calls its method). <i>For:</i> one of the two ways an interaction can arise; governs build/compile order. <i>When:</i> drawn as a directed arrow; in IA still treated as bidirectional because changing a callee impacts its callers.	ImpactAnalysis p.5, p.9
Coordination	<i>What:</i> two classes share coding/schedule/dataflow (e.g. wired together inside a third class's method body). <i>For:</i> captures interactions a dependency-only view misses, so IA doesn't undercount. <i>When:</i> e.g. <code>b.paint(a.get())</code> links <code>a</code> and <code>b</code> though neither references the other.	ImpactAnalysis p.5, p.8

Term	Definition	Source
Interaction graph $G=(X, I)$	<i>What:</i> X = set of classes (nodes), I = set of (undirected) interactions (edges). <i>For:</i> the data structure that makes IA an algorithm — "find impact" becomes "explore the graph." <i>When:</i> built first, then flood-filled by the marking loop.	ImpactAnalysis p.6
Neighborhood $N(A)$	<i>What:</i> $\{ B \mid (A, B) \in I \}$ — all classes one edge from A . <i>For:</i> the unit the algorithm expands by; defines the inspection frontier. <i>When:</i> when A becomes CHANGED/PROPAGATING, its BLANK neighbors $N(A)$ are marked NEXT.	ImpactAnalysis p.6
BLANK / NEXT / CHANGED / UNCHANGED	<i>What:</i> inspection marks — never inspected / scheduled / impacted / not impacted. <i>For:</i> the bookkeeping (a per-class state machine) that drives and terminates the loop; NEXT is the work queue, CHANGED expands the frontier, UNCHANGED closes a branch. <i>When:</i> each class moves BLANK→NEXT→{CHANGED, UNCHANGED,...} exactly once.	ImpactAnalysis p.11
PROPAGATING	<i>What:</i> a "mailman" class that relays change between neighbors without itself needing new code. <i>For:</i> lets the walk continue <i>through</i> pure conduits so impact on the far side isn't missed; without it the estimate would be too small. <i>When:</i> mark it PROPAGATING, then still push its BLANK neighbors to NEXT.	ImpactAnalysis p.10, p.12
INSPECTED	<i>What:</i> mark for an inspected-but-not-changed class in the interactive IA variant (in place of UNCHANGED at the loop-back point). <i>For:</i> records "a human looked and it's fine," closing the branch without expanding. <i>When:</i> used in the Computer/Programmer swimlane version.	ImpactAnalysis p.16
IA criteria	<i>What:</i> required effort vs. clarity of resulting code. <i>For:</i> choosing <i>which</i> candidate location to implement a change in, not just finding impact. <i>When:</i> applied when a change can be made in more than one place (e.g. Fahrenheit→Celsius); the two often conflict (short- vs. long-term).	ImpactAnalysis p.15
Process model / enactment / performance / plan	<i>What:</i> blueprint / the actual run with deviations / the collected measures / the expected future performance. <i>For:</i> separating intended-vs-actual-vs-measured-vs-predicted, which is what makes process improvement possible. <i>When:</i> the model→enact→measure→plan cycle around any process.	IntroSwProcesses p.5–6
Granularity hierarchy	<i>What:</i> lifecycle → stage → process → task → subtask/phase → step/action. <i>For:</i> a shared vocabulary that places each term at the right scale and shows how coarse activities nest fine ones. <i>When:</i> a "software change" is a task, "concept location" a phase, "inspect a class" a step.	IntroSwProcesses p.4
SIP	<i>What:</i> Solo Iterative Process — one programmer repeats changes, adding functionality one step at a time. <i>For:</i> the foundational iterative process every team process builds on; lets even a solo dev plan and meet obligations. <i>When:</i> individual maintenance and the course labs.	IntroSwProcesses p.7
Product backlog	<i>What:</i> the change requests — the vision for the software's future (bugs, demands, ideas). <i>For:</i> the single prioritized source of all work to be done. <i>When:</i> users feed it; iteration backlogs are drawn from it each cycle.	IntroSwProcesses p.9
Iteration backlog	<i>What:</i> the subset of the product backlog selected for the current iteration. <i>For:</i> the committed scope of one cycle — what the team will actually do next. <i>When:</i> extracted at the iteration meeting.	IntroSwProcesses p.10; TeamProcesses p.6
Baseline	<i>What:</i> a committed, integrated version produced at the end of a change/shift. <i>For:</i> keeps the codebase continuously in a known-good state and bounds lost work. <i>When:</i> scheduled at regular intervals; postponing is discouraged.	IntroSwProcesses p.10, p.26
Total time / Clean time	<i>What:</i> total includes interruptions; clean time = end – start – interruptions (focused work time). <i>For:</i> clean time predicts future effort independent of how the day was chopped up; total time is for scheduling. <i>When:</i> computed per step from the time log.	IntroSwProcesses p.14
LOC / KLOC / MLOC	<i>What:</i> lines-of-code size measures. <i>For:</i> a cheap, common size proxy (e.g. denominator of defect density), but inaccurate for work/value. <i>When:</i> quote only 1–2 significant digits (23 KLOC, not 23,418 LOC).	IntroSwProcesses p.16–17

Term	Definition	Source
Defect density	<i>What:</i> defects per KLOC — ~2.0 good, ~0.1 avionics (cutting edge). <i>For:</i> the standard quality metric; normalizes defect counts by size so systems are comparable. <i>When:</i> computed from the defect log and program size.	IntroSwProcesses p.19
Analogy / Decomposition	<i>What:</i> estimate phases from similar past phases / break a change into phases and sum. <i>For:</i> the two ways to produce an effort estimate; analogy is fast (needs a precedent), decomposition works without one (errors may cancel). <i>When:</i> during planning of a change.	IntroSwProcesses p.23
Epic	<i>What:</i> a large change that should be divided into smaller, more uniform changes. <i>For:</i> dividing epics makes changes alike and so more predictable (better analogy-based estimates). <i>When:</i> during tasking, before estimation.	IntroSwProcesses p.24
Release backlog table	<i>What:</i> per-change remaining-effort snapshots over time. <i>For:</i> a tracking dashboard that reveals on-time / late / incomplete release early. <i>When:</i> re-estimated and re-columned at each baseline; read the bottom two rows for the trend.	IntroSwProcesses p.28–31
AIP	<i>What:</i> Agile Iterative Process — small/medium team, consensus, no specialization. <i>For:</i> low-overhead, flexible development where consensus is cheap. <i>When:</i> small trusted teams; micro-cycle DEVELOP→REACT→MODIFY, no commit gate.	TeamProcesses p.4
DIP	<i>What:</i> Directed Iterative Process — manager-directed, specialized roles, scales large. <i>For:</i> coordinating large teams/systems where consensus and "everyone does everything" break down. <i>When:</i> large projects with developers, testers, architect, product/process managers.	TeamProcesses p.10
CIP	<i>What:</i> Centralized/Safeguarded Iterative Process — DIP plus a "permission to commit" gate. <i>For:</i> protecting quality/architecture when bad commits are unaffordable. <i>When:</i> large or quality-critical projects and open source; commits approved by architects/code owners.	TeamProcesses p.15–16
Agile Manifesto	<i>What:</i> 2001, 17 authors; 4 value statements (individuals/interactions, working software, customer collaboration, responding to change). <i>For:</i> the value system AIP embodies; states priorities, not absolutes (left valued <i>more</i> , right still valued). <i>When:</i> invoked to justify agile practices.	TeamProcesses p.8
Architect (role)	<i>What:</i> preserves architecture constraints; approves/disapproves commits. <i>For:</i> protecting the system's structural integrity against erosion by many independent changes. <i>When:</i> staffs the commit gate in DIP/CIP.	TeamProcesses p.13
Product vs. Process manager	<i>What:</i> strategic (what/why to build) vs. task assignment/process control (how/when). <i>For:</i> separating business direction from workflow management. <i>When:</i> both attached to the backlog/meetings in DIP/CIP.	TeamProcesses p.14
Code ownership	<i>What:</i> programmers specialize in parts of the code, each "owning" a subsystem. <i>For:</i> deep familiarity and accountability (staffs CIP's gate); risk is coordination bottlenecks. <i>When:</i> DIP/CIP/open source — the opposite of CI's "team owns the code."	TeamProcesses p.17
Continuous Integration (CI)	<i>What:</i> merging all working copies to a shared mainline several times a day, each integration verified by an automated build. <i>For:</i> killing integration hell — many small safe merges instead of one big painful one. <i>When:</i> triggered automatically on every commit (CILab: GitHub Actions + Maven).	Continuous Integration p.2; CILab p.1
Self-testing build	<i>What:</i> a build that runs its own (esp. unit) tests — best method for verifying builds. <i>For:</i> makes a green build mean <i>working</i> software, not just compiling software. <i>When:</i> unit tests preferred (fast, focused) so it stays fast enough to run on every commit.	Continuous Integration p.9
Build server	<i>What:</i> the final authority on stability/quality; only gets code from the repo. <i>For:</i> a neutral, reproducible arbiter that kills the "works on my machine" excuse. <i>When:</i> every commit; disputes resolved by what it does, not any laptop.	Continuous Integration p.6

Term	Definition	Source
Static analysis / Coverage	<i>What:</i> automated quality — find bugs/compliance without running code (Findbugs, PMD, Checkstyle); measure how much code tests exercise (Cobertura, Emma). <i>For:</i> catching defects and quality decay tests can't, and surfacing untested code. <i>When:</i> run automatically on each build (cost: lots of CPU, hence "keep it fast").	Continuous Integration p.10, p.13–14

Additional terminology — meetings, logs, measures, lab artifacts

Term	Definition	Source
Time log	<i>What:</i> the per-step record of an iteration — Start, End, number of interruptions, interruption minutes, Step (Pri/Ini/CL/IA/Ref/Ex/Act/Base), Comments. <i>For:</i> making the enactment observable; the raw data from which clean time and phase averages are derived. <i>When:</i> one row filled per step as you work.	IntroSwProcesses p.13
Defect log	<i>What:</i> per-defect record — Found (Date/Time/Task), Location, Description, Origin (Date/Task), Fixed. <i>For:</i> root-cause feedback — linking the symptom's discovery phase to the phase that introduced it. <i>When:</i> a row added when a defect is found; Origin/Fixed filled in as learned (entries may be -- or ?).	IntroSwProcesses p.20
Log = raw data	<i>What:</i> the principle that logs can become large and are <i>inputs</i> , not answers — you extract weekly clean-time summaries, average concept-location time, faster/slower trends, recurring SIP exceptions. <i>For:</i> turning fine-grained records into planning signals.	IntroSwProcesses p.15
Function points	<i>What:</i> a program-size measure based on delivered functionality; correlates with LOC but is harder to compute. <i>For:</i> a more design-level size proxy than raw lines.	IntroSwProcesses p.18
Number of methods / classes / files	<i>What:</i> size measures <i>even less accurate</i> than LOC. <i>For:</i> convenient counts, weak proxies — know they rank <i>below</i> LOC in accuracy.	IntroSwProcesses p.18
Defect (definition)	<i>What:</i> incorrect computations or premature termination. <i>For:</i> the unit counted by defect density and logged in the defect log.	IntroSwProcesses p.19
Tasking	<i>What:</i> shaping changes so they are more alike — narrow range of size, divide large changes ("epics") into smaller ones. <i>For:</i> predictability; similar tasks give reliable analogy data.	IntroSwProcesses p.24
Release plan	<i>What:</i> a commitment to release certain new functionality on a certain date, involving business considerations. <i>For:</i> planning makes sure that promise is realistic.	IntroSwProcesses p.27
Iteration meeting	<i>What:</i> the per-iteration checkpoint — assess the current product state (all stakeholders, technical + business viewpoints) and plan the next iteration; output = iteration backlog extracted from the product backlog. <i>When:</i> at the boundary between iterations.	TeamProcesses p.6
Daily meeting	<i>What:</i> daily team sync — problems/challenges, consensus on progress, daily assignment of change requests, clarify ambiguities, identify refactoring needs, early warning when anything goes wrong. <i>For:</i> high-frequency coordination of parallel changes.	TeamProcesses p.7
DEVELOP → REACT → MODIFY	<i>What:</i> AIP's micro-cycle — build, react to feedback, modify, repeat. <i>For:</i> the loop embodiment of "responding to change."	TeamProcesses p.4
Permission to commit	<i>What:</i> CIP's safeguard — developers' parallel changes need approval from Architects and Code owners before reaching the shared codebase. <i>For:</i> the gate that defines CIP versus DIP.	TeamProcesses p.16

Term	Definition	Source
Specialization	<i>What:</i> assigning people distinct roles (developer, tester, architect, managers, owners). <i>For:</i> "specialization increases effectiveness"; absent in AIP (programmers hold only the programmer role), present in DIP/CIP.	TeamProcesses p.4, p.10, p.12
Open source development	<i>What:</i> safeguarded + code ownership + a wide community of developers of variable skills — all three characteristics together. <i>For:</i> CIP-style control applied to an open contributor pool.	TeamProcesses p.18
System test	<i>What:</i> end-to-end test; often takes minutes to hours to run. <i>For:</i> thorough whole-system verification — too slow to be the per-commit build check.	ContinuousIntegration p.9
Unit test	<i>What:</i> fast (no database or file system), focused (pinpoints problems). <i>For:</i> the best method for verifying builds.	ContinuousIntegration p.9
Hotspot	<i>What:</i> an area of low testing and high complexity found by unit-test analysis (SONAR). <i>For:</i> pointing at the riskiest, least-verified code.	ContinuousIntegration p.10
Environments based on stability	<i>What:</i> server environments that model code stability; code is promoted to stricter environments as quality improves. <i>For:</i> CI principle 1 — a risk gradient before production.	ContinuousIntegration p.3–4
"If it hurts, do it more often"	<i>What:</i> the agile principle justifying frequent builds — difficult activities become more straightforward done more frequently; reduces time between defect introduction and removal.	ContinuousIntegration p.7
GitHub Actions workflow (*.yml)	<i>What:</i> the file at <code><YOUR_PROJECT>/github/workflows/</code> that tells GitHub Actions CI what to do — configured in the lab to build each pull request with Maven and execute tests automatically.	CILab p.1
.maven-settings.xml / GitHub Packages	<i>What:</i> a settings file in the project root enabling shared jars from the GitHub Packages registry ("Working with the Apache Maven registry"). <i>For:</i> dependency resolution from a shared registry rather than local machines.	CILab p.1
Figure 7.9 [Raj13]	<i>What:</i> the Rajlich-textbook activity diagram of the IA process the AnalysisLab classwork instructs you to follow to find the Estimated Impact Set.	AnalysisLab p.1
Table 1 (AnalysisLab portfolio)	<i>What:</i> the portfolio deliverable — columns Package name / # of classes / Comments — listing every package visited during impact analysis, with comments on what you learned about each package and how it contributes to your feature.	AnalysisLab p.1–2
[Bec99] / [Boo] / [Tho]	<i>What:</i> the CILab's three references — Kent Beck, <i>Embracing change with extreme programming</i> (1999); Grady Booch, <i>Object Oriented Design: With Applications</i> ; ThoughtWorks, <i>Continuous Integration</i> (sic). <i>For:</i> the CI history facts (Booch coined CI 1991; XP advocated integrating tens of times per day).	CILab p.1

Common Pitfalls / Gotchas

- **Initial vs. estimated impact set are different sets.** Concept Location gives the *initial* set; IA *expands* it into the *estimated* set. Saying "IA finds the concept" confuses IA with Concept Location (ImpactAnalysis p.3–4).
- **IA walks the interaction graph, not the dependency graph.** Because interactions propagate change **both** ways ($A \leftrightarrow B$), you cannot just follow dependency arrows one direction (ImpactAnalysis p.5, p.9).

- **Don't forget PROPAGATING (mailman) classes.** A class can need *no* code change yet still force its neighbors to be inspected because change *passes through* it. Marking it UNCHANGED would wrongly stop propagation (ImpactAnalysis p.10).
- **The algorithm only seeds CHANGED from concept location, then expands neighbors of CHANGED/PROPAGATING classes** — UNCHANGED/INSPECTED classes do **not** extend the frontier. Mixing this up makes the loop never terminate or terminate too early (ImpactAnalysis p.12).
- **The "estimated" in estimated impact set means it's a prediction** — it can be wrong; Verification (the right-hand bar of the process diagram) is what catches mis-estimates (ImpactAnalysis p.3–4).
- **The two IA criteria genuinely conflict.** Least-effort (patch the UI) often hurts clarity (scattered logic). Don't present them as if minimizing effort is always correct (ImpactAnalysis p.15).
- **Granularity confusion.** "Software change" is a **task**; "concept location"/"actualization" are **phases**; "inspection of a class" is a **step**; SIP/AIP/DIP are **processes**. Exam questions test whether you can place a term at the right level (IntroSwProcesses p.4).
- **Model ≠ enactment.** The process model is the blueprint; the enactment always has deviations/exceptions (e.g. the "install new Bugzilla" interruption). Performance is the *measurements*, plan is the *expected future* (IntroSwProcesses p.5–6).
- **Clean time excludes interruptions** — `end - start` alone is *total* time, not clean time (IntroSwProcesses p.14).
- **LOC is inaccurate and only 1–2 significant digits are meaningful.** Don't quote "23,418 LOC" as if precise — it's "23 KLOC" (IntroSwProcesses p.16–17).
- **Defect density numbers are easy exam targets:** ~2.0/KLOC (good), ~0.1/KLOC (avionics/Space Shuttle). Don't swap them (IntroSwProcesses p.19).
- **Release backlog: "remaining effort" falling is good, but "total effort to reach goal" rising means scope is inflating.** A release can finish *late* (effort reaches 0 at 475h vs. planned 330h) or *incomplete* (ship without items 11–12) (IntroSwProcesses p.29–31).
- **AIP vs. DIP vs. CIP confusion.** AIP = consensus, *no* specialization, small/medium. DIP = manager-directed, specialized roles, scales large. CIP = like DIP but adds the **"permission to commit"** safeguard via architects/code owners. The commit gate is CIP's signature (TeamProcesses p.4, p.10, p.16).
- **The Agile Manifesto keeps the right side too.** "X over Y" means X is valued *more*, not that Y is worthless ("while there is value in the items on the right, we value the items on the left more"). All four statements: individuals/interactions, working software, customer collaboration, responding to change (TeamProcesses p.8).
- **CI: the build server, not your laptop, is the source of truth.** "It builds on my box" is explicitly the anti-pattern CI exists to defeat — the repo is the source of record and the build server settles disputes (ContinuousIntegration p.6).
- **"Build every commit" is justified by "if it hurts, do it more often"** and by shrinking the defect-introduction-to-removal gap — not by ceremony (ContinuousIntegration p.7).
- **Unit tests vs. system tests:** unit tests are fast/focused/no I/O and are the *best* for verifying builds; system tests are end-to-end and slow. Keep the build fast (ContinuousIntegration p.9, p.11).
- **CI lab history facts:** Booch coined "CI" (1991) but didn't advocate daily integration; XP/Beck did. Easy to misattribute (CILab p.1).

More easily-confused distinctions

- **Pri and Ex are log rows, not change-process phases.** The eight-phase pipeline is Initiation → Concept Location → Impact Analysis → Prefactoring → Actualization → Postfactoring → Conclusion (+ cross-cutting Verification) (ImpactAnalysis p.3). *Pri* (prioritize) is backlog management before initiation; *Ex* (exception) is an enactment deviation — e.g. "Install new version of Bugzilla" (IntroSwProcesses p.11).

- **[UNCHANGED] guard vs. "Mark class as INSPECTED" box.** On the p.12 activity diagram the branch is guarded [UNCHANGED] but the action box says "Mark class as INSPECTED"; on p.16 the guard itself is [INSPECTED]. Same outcome, two labels — don't treat them as different states (ImpactAnalysis p.12, p.16).
- **"Remaining effort" ≠ "total effort needed to reach the goal."** The second equals *hours already worked + remaining effort* (100 + 340 = 440 after 100 hours). Remaining effort can fall while total-to-goal rises (IntroSwProcesses p.29–30).
- **Function points are harder to compute than LOC but correlate with it; method/class/file counts are even less accurate than LOC** — keep the accuracy ordering straight: function points ≈ LOC-correlated, LOC inaccurate, counts of methods/classes/files worse still (IntroSwProcesses p.16–18).
- **Know the defect definition, not just the densities:** a defect is "incorrect computations, premature termination" (IntroSwProcesses p.19).
- **Never mark the mailman CHANGED.** The propagating class itself needs no new code — its mark is PROPAGATING, and only its *neighbors* get queued (ImpactAnalysis p.10, p.12).
- **AIP's "no specialization" is about programmers, not the whole org chart.** The AIP model still attaches a **Product Manager** and a **Process Manager** to the backlog/meetings; the claim is that *developers have only the programmer role* (TeamProcesses p.4–5).
- **Who builds, who gates:** in the DIP model **Testers** handle Build (TeamProcesses p.11); in CIP, Testers still build but the distinctive element is the **permission-to-commit** gate staffed by **Architects and Code owners** (TeamProcesses p.16).
- **Open source has three slide-listed characteristics** — safeguarded, code ownership, wide community of variable skills. Listing only "it's open" misses the point (TeamProcesses p.18).
- **"Fast" for unit tests has a concrete meaning:** *no database or file system* — that, plus being focused enough to pinpoint problems, is why they are the best method for verifying builds (ContinuousIntegration p.9).
- **Tool-to-category mapping is exam bait:** Selenium and FitNesse are *test frameworks* (browser and acceptance testing, ContinuousIntegration p.12), not static analyzers; **Cobertura** is *coverage* (p.10, p.14), not static analysis; **Sonar** appears both in the static-analysis list (p.13) and as the hotspot-finding unit-test analyzer (p.10); **FXCop** and **NUnit/MSTest** are the .NET entries of their lists (p.12–13).
- **Deck-title nuance:** the first deck's own title is "**Impact and Feature Analysis**" (ImpactAnalysis p.1) — if an exam references "the feature analysis lecture," it is this deck.
- **The two IA criteria are about where to make a change, not whether.** Effort vs. clarity weighs alternative *locations* (sensor conversion vs. display) for an already-decided change (ImpactAnalysis p.14–15).
- **Baseline scheduling is regular and non-postponable** — "every day at the end of the shift" or "every other day after a change is finished"; *postponing is not recommended* (IntroSwProcesses p.26). Don't confuse baselining (commit/integrate a known-good state) with releasing (business-facing delivery, p.27).
- **Estimates in the release backlog only ever grew in the worked tables** (inventory +20, multiple prices +10, cashier sessions +45, multiple line items +35, multiple cashiers +25, credit payment +10) — work reveals hidden complexity; the tables never show an estimate shrinking (IntroSwProcesses p.29–30).

Exam Focus

Very likely to be asked: - **Run the IA algorithm by hand** on a given interaction graph: mark BLANK, seed the concept-location class as CHANGED, propagate NEXT to neighbors, classify each as CHANGED/PROPAGATING/UNCHANGED, and report the **estimated impact set**. Know the loop and its

termination (ImpactAnalysis p.11–12, p.16; AnalysisLab Fig 7.9). - **Define and contrast** initial vs. estimated impact set; dependency vs. interaction; dependency diagram vs. interaction diagram (ImpactAnalysis p.3–9). - **Explain propagating/mailman classes** with the John–mailman–Paul story and why they matter (ImpactAnalysis p.10). - **The two IA criteria** (effort vs. clarity) and the short/long-term conflict, illustrated by Fahrenheit→Celsius (ImpactAnalysis p.14–15). - **Place a term in the granularity hierarchy** and identify where IA / concept location / class inspection / SIP sit (IntroSwProcesses p.4). - **Distinguish model / enactment / performance / plan** (IntroSwProcesses p.5–6). - **SIP**: what it is, why a solo programmer still needs a process, its workproducts and model loop (IntroSwProcesses p.7–10). - **Clean time computation** from a time log (IntroSwProcesses p.13–14). - **Defect density numbers** (~2.0 good, ~0.1 avionics) and what a defect log records (IntroSwProcesses p.19–20). - **Read a release backlog table** to decide on-time / late / incomplete and explain the two bottom rows (IntroSwProcesses p.28–31). - **Compare AIP, DIP, CIP** and list the team roles (developer, tester, architect, product/process manager, code owner); the CIP "permission to commit" gate (TeamProcesses p.4–17). - **The four Agile Manifesto statements** and its year (2001) / 17 authors (TeamProcesses p.8). - **The five CI principles** and the key slogans ("build every commit," "if it hurts do it more often," "team owns the code," "keep it fast," "build server is the final authority") (ContinuousIntegration p.2–11). - **Why self-testing builds / unit tests**, and the "3+ methods → >90% defect removal" / "individuals <50% at finding own bugs" figures (ContinuousIntegration p.8–9). - **Name tooling categories**: test frameworks (JUnit, Selenium, ...), static analysis (Findbugs, PMD, Checkstyle, Sonar, ...), coverage (Cobertura, Emma, Clover) (ContinuousIntegration p.12–14).

One-line summaries to memorize: - IA = expand the **initial** impact set into the **estimated** impact set by walking the **interaction** graph, marking classes until no **NEXT** remains. - A **propagating** class relays change without changing itself. - **Clean time = end – start – interruptions**. - Good software ≈ **2 defects/KLOC**; avionics ≈ **0.1 defects/KLOC**. - AIP = consensus/no roles; DIP = manager-directed/specialized; CIP = DIP + commit safeguard. - CI: **integrate many times a day, build every commit, self-test the build, keep it fast, repo+build-server are the source of truth**.

Rapid-recall fact sheet — every number on the Lecture 3 slides

Number	Fact	Source
6	IA marks in total: BLANK, NEXT, CHANGED, UNCHANGED (verdict table) + PROPAGATING, INSPECTED (algorithm diagrams)	ImpactAnalysis p.11–12, p.16
2	IA decision criteria: required effort; clarity of the resulting code	ImpactAnalysis p.15
2	Alternative change locations in the temperature example: sensor conversion; user display	ImpactAnalysis p.14
6	Granularity levels: lifecycle, stage, process, task, subtask/phase, step/action	IntroSwProcesses p.4
4	Forms of process: model, enactment, performance, plan	IntroSwProcesses p.5–6
8	Rows in the SIP enactment example: Pri, Ini, CL, IA, Ref, Ex, Act, Base	IntroSwProcesses p.11
4	Classes in the enactment's estimated impact set ("Estimated set has 4 classes" / "IA — 4 classes")	IntroSwProcesses p.11, p.13
19 min	Clean time of the IA step: 9:23 – 8:52 = 31 min, minus 12 min interruptions	IntroSwProcesses p.13–14
15 min	Clean time of an American-football quarter (the clean-time analogy)	IntroSwProcesses p.14

Number	Fact	Source
2	Regression faults found at Base and added to the backlog	IntroSwProcesses p.11, p.13
1–2	Significant digits of LOC that are meaningful (900 LOC, 23 KLOC, 3.2 MLOC)	IntroSwProcesses p.17
~2.0 /KLOC	Defect density of good-quality software	IntroSwProcesses p.19
~0.1 /KLOC	Defect density of avionics software — cutting edge; NASA Space Shuttle	IntroSwProcesses p.19
12	Changes in the release backlog (initial ... check payment)	IntroSwProcesses p.29
330 → 340 → 180 → 70 → 0	Remaining-effort trajectory of the late release	IntroSwProcesses p.30
330 → 440 → 465 → 475 → 475	Total-effort-to-goal trajectory of the late release	IntroSwProcesses p.30
0 / 100 / 285 / 405 / 475	Snapshot hours in the late-release table (incomplete release stops at 405)	IntroSwProcesses p.30–31
70	Effort cut in the incomplete release = credit payment (50) + check payment (20)	IntroSwProcesses p.31
2001 / 17	Agile Manifesto year / original authors	TeamProcesses p.8
4	Agile Manifesto value statements	TeamProcesses p.8
3	Open source characteristics: safeguarded, code ownership, wide variable-skill community	TeamProcesses p.18
5	Principles of Continuous Integration	ContinuousIntegration p.3
< 50%	Individual programmers' efficiency at finding their own bugs	ContinuousIntegration p.8
3+ / > 90%	Quality methods to combine / resulting defect removal	ContinuousIntegration p.8
3	Most effective quality methods: design inspections, code inspections, testing	ContinuousIntegration p.8
1991	Booch first proposed the term CI (without advocating several-times-a-day integration)	CILab p.1
1999	Kent Beck, <i>Embracing change with extreme programming</i>	CILab p.1
tens/day	Integration frequency XP advocated ("perhaps as many as tens of times per day")	CILab p.1
5	Numbered classwork steps in the CILab	CILab p.1

Likely exam question formats for this lecture

- **Trace task:** given a small interaction graph and a seed class, run the marking loop and report the estimated impact set, listing the mark of every class at the end (ImpactAnalysis p.12–13, p.16; AnalysisLab p.1).
- **Table-reading task:** given a time log row, compute clean time; given a release-backlog table, classify the scenario as on-time / late / incomplete and explain via the two bottom rows (IntroSwProcesses p.13–14,

p.28–31).

- **Classification task:** place a term (software change, concept location, SIP, inspection of a class) at the right granularity level; or identify which form (model/enactment/performance/plan) a shown artifact is (IntroSwProcesses p.4–6).
- **Compare/contrast essay:** AIP vs. DIP vs. CIP, or code ownership vs. "team owns the code," or unit vs. system tests in a self-testing build (TeamProcesses p.4–18; ContinuousIntegration p.5, p.9).
- **Justify-the-practice question:** why build every commit ("if it hurts, do it more often"; shrink defect-introduction-to-removal time), why self-testing builds (<50% self-detection; 3+ methods >90%), why keep the build fast (ContinuousIntegration p.7–9, p.11).
- **Tool-category matching:** match JUnit/NUnit/MSTest/Selenium/FitNesse, Checkstyle/CodeScanner/DRY/Crap4j/Findbugs/PMD/Fortify/Sonar/FXCop, and Emma/Cobertura/Clover/GCC-GCOV to test frameworks / static analysis / coverage (ContinuousIntegration p.12–14).
- **History fact-check:** who coined CI (Booch, 1991) vs. who advocated multiple daily integrations (XP) (CILab p.1).

Source Map

Deck / Lab	Pag es	Sections covered
ImpactAnalysis.pdf	p.1–2	Title / section divider
	p.3	IA definition; position in the change process; initial impact set; produces estimated impact set
	p.4	Initial vs. estimated impact set (diagram)
	p.5	Class interactions; dependency vs. coordination; bidirectional propagation
	p.6	Interaction graph $G=(X, I)$; neighborhood $N(A)$
	p.7	Neighborhood of Item (Store/Inventory/Item/Price/Cashiers/CashierRecord)
	p.8	Coordination code example (<code>b.paint(a.get())</code>)
	p.9	Dependency diagram vs. interaction diagram
	p.10	Propagating (mailman) classes — John/mailman/Paul
	p.11	Status marks table (BLANK/CHANGED/UNCHANGED/NEXT)
	p.12	IA algorithm incl. propagating classes (activity diagram)
	p.13	IA worked example trace (Concept location → Step 1–5 → Iteration 6)
	p.14	Alternatives in change (Fahrenheit→Celsius)
	p.15	The two criteria (effort vs. clarity; short/long-term conflict)
	p.16	Interactive IA (Computer vs. Programmer swimlanes; INSPECTED mark)
IntroSwProcesses.pdf	p.1	Why study processes; Good process → Good product
	p.2	Variability (team; system)
	p.3–4	Granularity; granularity hierarchy table

Deck / Lab	Pag es	Sections covered
	p.5–6	Forms of process: model, enactment, performance, plan
	p.7–8	SIP definition; why SIP for a solo programmer
	p.9	Workproducts (product backlog, code, docs)
	p.10	SIP model diagram
	p.11	Enactment of SIP (Pri/Ini/CL/IA/Ref/Ex/Act/Base table)
	p.12–15	Measuring SIP; time log; total vs. clean time; log as raw data
	p.16–18	Program size: LOC/KLOC/MLOC; function points; other measures
	p.19–20	Code defects; defect density; defect log
	p.21–24	Planning; repetitions; analogy & decomposition; tasking/epics
	p.25	Tasking example (sales coupons)
	p.26–27	Baselines; release plan
	p.28–31	Release backlog tables: original / after 100h / late / incomplete
TeamProcesses.pdf	p.1–2	Title; why teams
	p.3–4	AIP intro; AIP characteristics (consensus, no specialization; DEVELOP→REACT→MODIFY)
	p.5	Model of AIP
	p.6	Iterations / iteration meeting / iteration backlog
	p.7	Daily meeting
	p.8	Agile Manifesto (2001, 17 authors, 4 values)
	p.9–10	DIP intro and characteristics (manager-directed, specialized, scales)
	p.11	Model of DIP
	p.12	Roles: developers, testers, specialization increases effectiveness
	p.13	Architect role (preserve architecture; approve/disapprove commits)
	p.14	Management (product vs. process managers)
	p.15–16	CIP intro; Model of CIP (permission to commit)
	p.17	Code ownership (specialization; coordination problem)
	p.18	Open source development (safeguarded; code ownership; wide community)
ContinuesIntegration.pdf	p.1	Title

Deck / Lab	Pag es	Sections covered
	p.2	CI definition and value; CI cycle (Commit→Build→Test→Report)
	p.3	Five principles of CI
	p.4	Environments based on stability (Dev→Test→Stage→Prod)
	p.5	Commit frequently / build every commit (unit tests; team owns code)
	p.6	"Builds on my box"; repo = source of record; build server = final authority
	p.7	Why build every commit ("if it hurts do it more often"; automate build)
	p.8	Add testing to build (<50% self bug-find; 3+ methods → >90% removal; inspections + testing)
	p.9	Self-testing builds (system vs. unit tests)
	p.10	Automated quality (static analysis Findbugs/PMD/Checkstyle; coverage Cobertura; SONAR hotspots)
	p.11	Build server hardware ("KEEP IT FAST")
	p.12	Test frameworks (JUnit/NUnit/MSTest/Selenium/FitNesse)
	p.13	Static analysis tools (Checkstyle/CodeScanner/DRY/Crap4j/Findbugs/PMD/Fortify/Sonar/FxCop)
	p.14	Code coverage tools (Emma/Cobertura/Clover/GCC-GCOV) + coverage view example
AnalysisLab1.pdf	p.1	IA definition; objective (static+dynamic analysis); classwork = Fig 7.9 algorithm; portfolio table of packages/classes
	p.2	Table 1 (package name / # classes / comments)
CILab.pdf	p.1	CI definition + history (Booch 1991, XP/Beck); objectives; classwork (GitHub Actions <code>.yaml</code> + Maven build/tests; <code>.maven-settings.xml</code>); references [Bec99][Boo][Tho]

Deck and lab metadata (titles, authorship, numbering quirks)

- **ImpactAnalysis.pdf** — the slide deck's own title is "**Impact and Feature Analysis** (SB5-MAI)" by **Jan Corfixen Sørensen, University of Southern Denmark** (ImpactAnalysis p.1); p.2 is a section divider repeating "Impact and Feature Analysis." If exam material refers to "the impact and feature analysis slides," it means this deck.
- **IntroSwProcesses.pdf** — the first slide is headed "**12 Introduction to software processes**" (IntroSwProcesses p.1); the leading "12" matches a textbook chapter-numbering convention (the deck reads as the Rajlich Ch. 12 slide set — an inference from the heading; the deck pages themselves carry only slide numbers 1–31). Unlike the other three decks, it has no separate title/author slide — content starts immediately.
- **TeamProcesses.pdf** — titled "**Team Iterative Processes** (SB5-MAI)," same author and university (TeamProcesses p.1). Internal section dividers split the deck into its three process families: p.3 "Agile Iterative Processes (AIP)," p.9 "Directed Iterative Processes (DIP)," p.15 "Centralized Iterative Processes (CIP)."
- **ContinuesIntegration.pdf** — titled "**Continuous Integration** (SB5-MAI)," same author (ContinuousIntegration p.1). Note the filename spells "Continues" while the slide title spells "Continuous"; this guide cites it as `ContinuousIntegration`.
- **AnalysisLab1.pdf** — handout tagged "[AnalysisLab]," titled "**Impact Analysis Lab**"; p.1 carries Introduction/Objectives/Classwork/Portfolio Work, p.2 carries the empty **Table 1** ("The list of all the packages visited during impact analysis") with columns *Package name / # of classes / Comments* (AnalysisLab p.1–2).

- **CILab.pdf** — handout tagged "[CILab]," titled (sic) "**Impact Continues Integration Lab**" — a single page with Introduction/Objectives/Classwork/References (CILab p.1).
-

Slide-by-Slide Walkthrough — ImpactAnalysis.pdf (16 slides)

This walkthrough restates each slide of the deck in order, with the slide's literal content plus what to take from it. Use it to reconstruct the deck mentally before the exam.

Slides 1–2 — Title and section divider

Slide 1: "Impact and Feature Analysis (SB5-MAI)," Jan Corfixen Sørensen, University of Southern Denmark. Slide 2 repeats the title as a section divider (ImpactAnalysis p.1–2). Takeaway: the deck's scope as titled is impact *and* feature analysis; the sixteen slides develop the impact half via the interaction-graph machinery.

Slide 3 — Impact analysis (IA) and the change-process pipeline

Four bullets beside the vertical process diagram: IA "determines the strategy and impact of change"; "classes identified in concept location are the initial impact set"; "class dependencies are analyzed, and impacted classes are added to the impact set"; it "produces estimated impact set." The diagram stacks Initiation, Concept Location, **Impact Analysis**, Prefactoring, Actualization, Postfactoring, Conclusion, with VERIFICATION written vertically along the right edge (ImpactAnalysis p.3). Takeaway: input = initial impact set (from Concept Location); operation = analyze dependencies and add impacted classes; output = estimated impact set; verification spans all phases.

Slide 4 — Initial and estimated impact set

A diagram-only slide titled "Initial and estimated impact set": the initial impact set sits inside the software, and arrows labelled with the IA expansion radiate outward to the larger estimated impact set region (ImpactAnalysis p.4). Takeaway: the initial set is a subset of the estimated set, and IA is the expansion between them.

Slide 5 — Class Interactions

"Two classes interact if the[y] have something in common," with the two flavors: "one depends on the other — there is a contract between them," and "they coordinate — they share the same coding, schedule, etc." Then the load-bearing property: "**interactions propagate change in both directions — from A to B or from B to A**" (ImpactAnalysis p.5). Takeaway: the bidirectionality bullet is the single most quotable line of the deck; it is why IA walks an undirected graph.

Slide 6 — Class Interaction Graph

The formalization: $G = (X, I)$ with X "set of classes" and I "set of interactions"; the **neighborhood of class A** defined as $N(A) = \{B \mid (A,B) \in I\}$ (ImpactAnalysis p.6). Takeaway: be able to write both definitions symbol-for-symbol; $N(A)$ is the frontier-expansion unit of the algorithm.

Slide 7 — Neighborhood of Item

A UML-style picture: `Store` at the top connected to `Inventory` and `Cashiers`; `Inventory` connected to `Item`; `Item` connected to `Price`; `Cashiers` connected to `CashierRecord`. The slide highlights `Item`'s

neighborhood (ImpactAnalysis p.7). Takeaway: $N(\text{Item}) = \{\text{Inventory}, \text{Price}\}$; the picture is the graph used by this guide's Worked Example A.

Slide 8 — Coordination

A code slide:

```
class C {  
  A a;      // gets the color code  
  B b;      // paints the screen  
  void foo() {  
    b.paint(a.get()); // dataflow between a and b  
  }  
};
```

The comment line is the lesson: `b.paint(a.get());` is a "dataflow between a and b" — a coordination interaction between `A` and `B` created inside `C`'s body, with neither `A` nor `B` referencing the other (ImpactAnalysis p.8). Takeaway: coordination edges live in *third-party* code; this is the canonical example to reproduce when asked "give an interaction that is not a dependency."

Slide 9 — Dependency diagram, Interaction diagram

A diagram-only slide contrasting the two notations for the slide-8 situation: the dependency diagram shows directed (dashed) arrows from `C` to `A` and `C` to `B`; the interaction diagram shows undirected lines `C — A`, `C — B`, and `A — B` (the coordination edge) (ImpactAnalysis p.9). Takeaway: the interaction diagram has *more* edges (it adds `A—B`) and *no* direction — both differences matter for IA correctness.

Slide 10 — Propagating class: Mailman

The full story as bulleted on the slide: John loaned money to Paul; John needs the money back — *his situation changed*. John writes a letter to Paul; the **mailman takes the letter from John to Paul**; Paul must take a part-time job — "a big change that propagated from John to Paul." John interacts with the mailman; the mailman interacts with Paul; "the change originated with John and propagates **through** the mailman to Paul" (ImpactAnalysis p.10). Takeaway: the mailman's own life is unchanged — the analogy's force is that an unchanged intermediary still carries change, hence the PROPAGATING mark.

Slide 11 — Process of Impact Analysis (the marks)

A four-row table defining the marks: **Blank** — "the class was never inspected and is not scheduled for an inspection"; **Changed** — "the programmers inspected the class and found that it is impacted by the change"; **Unchanged** — "the programmers inspected the class and found that it is not impacted by the change"; **Next** — "the class is scheduled for inspection" (ImpactAnalysis p.11). Takeaway: these are the verdict-level definitions; PROPAGATING and INSPECTED only appear once the algorithm diagrams arrive (p.12, p.16).

Slide 12 — IA including propagating classes (activity diagram)

The activity diagram's nodes, in order: "Create interaction diagram and mark all classes as BLANK" → "Mark the class located during concept location as CHANGED" → "Mark all BLANK neighbors as NEXT" → decision "Are there any classes marked as NEXT?" with **[No]** exiting and **[Yes]** leading to "Select a class marked as NEXT. What is the new mark for this class?" whose three guarded branches are **[UNCHANGED]** → "Mark class as INSPECTED," **[PROPAGATING]** → "Mark class as PROPAGATING," and **[CHANGED]** → "Mark class as CHANGED" — the latter two looping back through "Mark all BLANK neighbors as NEXT"

(ImpactAnalysis p.12). Takeaway: only CHANGED and PROPAGATING re-enter the neighbor-marking step; the [UNCHANGED] branch records INSPECTED and returns straight to the NEXT test.

Slide 13 — IA Example

A trace-only slide titled "IA Example": the original interaction graph, then the concept-location seeding, then Steps 1–5 and Iteration 6, with box shadings showing classes moving BLANK → NEXT (hatched) → CHANGED (dark) / UNCHANGED (U) (ImpactAnalysis p.13). Takeaway: practice reproducing such a trace by hand — this guide's Worked Example A is the equivalent exercise on the slide-7 graph.

Slide 14 — Alternatives in Software Change

The temperature scenario: a program displays temperature in Fahrenheit; the change request is to display it in Celsius. "Two separate locations deal with temperature": where "sensor data [is] converted to the temperature" and where "temperature [is] displayed to the user." "The change can be done in either place — impact analysis weights these alternatives" (ImpactAnalysis p.14). Takeaway: IA is not only discovery but *decision support* between candidate implementation sites.

Slide 15 — The Criteria

The two criteria: "**required effort of the change**" and "**clarity of the resulting code**." "Often, these two criteria contradict each other: it is easier to adjust the user interface; it is better to have all calculations of the temperature in one place." Conclusion: a "**conflict between short-term and long-term goals**" (ImpactAnalysis p.15). Takeaway: quote both criteria and the conflict line verbatim; the temperature example instantiates them.

Slide 16 — Interactive IA

The same activity diagram as slide 12 but split into **Computer** and **Programmer** swimlanes: the Computer creates the interaction diagram, marks BLANK, seeds CHANGED, marks BLANK neighbors NEXT, applies the marks, and runs the "any NEXT?" test; the Programmer performs "Select a class marked as NEXT. What is the new mark for this class?" with guards **[INSPECTED]**, **[PROPAGATING]**, **[CHANGED]** (ImpactAnalysis p.16). Takeaway: the human supplies only the judgement step; everything mechanical is tool work — IA tooling is human-in-the-loop, and here the not-impacted guard is literally labelled INSPECTED.

Slide-by-Slide Walkthrough — IntroSwProcesses.pdf (31 slides)

The deck opens directly with content (no title slide); its first heading reads "12 Introduction to software processes" (IntroSwProcesses p.1).

Slide 1 — Why study software processes

"Study of software processes is the **core of software engineering**." Two learning sources: **successful projects of the past** ("the processes that worked well" become "a prescription for the future projects") and **unsuccessful projects** ("find the problems that led to the failure"). Closing slogan: "**Good process → Good product**" (IntroSwProcesses p.1). Takeaway: processes are studied because they are transferable; the slogan is a standard short-answer line.

Slide 2 — Variability of processes

Processes vary along **Team** (organization, collaboration, skills) and **System** (technology, domain, size, and expected quality) (IntroSwProcesses p.2). Takeaway: two axes, seven listed sub-factors — this is why the course teaches a family of processes rather than one.

Slides 3–4 — Granularity and the granularity table

Slide 3: coarse-granularity processes "deal with long periods of time"; "software life span models are example of a process of very coarse granularity"; stages are also processes; the word "process" is "usually used for processes that fit within a single stage or few neighboring stages" (IntroSwProcesses p.3). Slide 4 is the six-row table — lifecycle (staged, waterfall) / stage (evolution, servicing) / process (SIP, AIP, DIP) / task (software change, acceptance testing) / subtask, phase (concept location, actualization) / step, action (inspection of a class) (IntroSwProcesses p.4). Takeaway: memorize the table with its examples; note "acceptance testing" is the *second* example at the task level and "waterfall"/"staged" at lifecycle level.

Slides 5–6 — Forms of process

Slide 5: **process model** = "prescription what the tasks should be and how should they fit together; a blueprint how to do things"; **enactment** = "the actual process in the project; inevitable deviations and exceptions from the process model" (IntroSwProcesses p.5). Slide 6: **performance** = "set of measures that an observer of an enacted process collects: time, cost, quality, ..."; **plan** = "expected future performance; decisions that the project stakeholders take; alternatives how to enact the process model" (IntroSwProcesses p.6). Takeaway: model and enactment on one slide, performance and plan on the next — four forms, intended/actual/measured/predicted.

Slides 7–8 — SIP and why SIP at all

Slide 7: SIP = "single programmer repeats software changes; functionality is added one step at a time"; repeated changes are the basis of "software evolution" and "software servicing, reengineering"; SIP "demonstrates characteristics shared by all iterative processes" (IntroSwProcesses p.7). Slide 8 poses the question "why should solo programmer follow a predefined process rather than react flexibly to challenges?" and answers: even solo programmers must "meet their obligations — fulfill their promises, pay their bills, plan the future, manage their own resources. SIP is the process that allows that" (IntroSwProcesses p.8). Takeaway: the rhetorical question-and-answer is itself exam material — flexibility without process gives no basis for commitments.

Slide 9 — Workproducts

Three workproducts: the **product backlog** ("change requests; represents the vision for the future of the software; bugs in the functioning of software; new demands and ideas"), **software code**, and **software documentation, etc.** (IntroSwProcesses p.9). Takeaway: the backlog's four-part description — change requests / vision / bugs / new demands and ideas — is fully enumerable.

Slide 10 — SIP model diagram

The model picture: **Users** send **change requests** into the **product backlog**; an **iteration backlog** is extracted; the solo programmer (labelled "Sol") performs **software changes**, producing a **code update**, then a **baseline**, then an **iteration/release**, with **delivery** flowing back to the users (IntroSwProcesses p.10). Takeaway: be able to redraw this loop; AIP/DIP/CIP are elaborations of exactly this picture with more people and meetings.

Slide 11 — Enactment of SIP

The eight-row table: 1 **Pri** "New change request arrives"; 2 **Ini** "Add Cashier Session"; 3 **CL** "CashierRecord"; 4 **IA** "Estimated set has 4 classes"; 5 **Ref** "Extract class Session"; 6 **Ex** "Install new version of Bugzilla"; 7 **Act** "Replace class Session"; 8 **Base** "2 regression faults added to backlog"; the table ends with "..." indicating the loop continues (IntroSwProcesses p.11). Takeaway: a complete one-iteration enactment including a deviation (Ex) — the bridge between the abstract model (slide 10) and measured reality (slide 13).

Slide 12 — Measuring SIP

Two bullets: "data indicate how the process is working" and "data serve as a foundation for future planning" (IntroSwProcesses p.12). Takeaway: the two *purposes* of measurement — monitoring and planning — quoted nearly verbatim in many exam answers.

Slide 13 — Time log

The worked log with columns Start / End / # / time (interruptions) / Step / Comments, rows as tabulated in Worked Example C: Pri 8:23–8:31; Ini 8:32–8:39 (1 interruption, 2 min); CL 8:42–8:52; IA 8:52–9:23 (2, 12); Ref 9:27–10:46 (3, 25); Ex 10:50–11:42 (2, 6); Act 1:23–2:17; Base 2:22–3:12 (3, 12). Slide annotations label the **process enactment**, **interruption**, **End**, and **Start** parts of the table (IntroSwProcesses p.13). Takeaway: this one artifact is simultaneously enactment record and performance data.

Slide 14 — Time (total vs. clean)

Total time vs. **clean time**, the American-football analogy ("15 minutes is the clean time of each quarter — total time includes all interruptions"), and the formula "**Clean time = end – start – time of interruptions**" (IntroSwProcesses p.14). Takeaway: formula + analogy; compute clean time for any log row on demand.

Slide 15 — Log = raw data

The log "can become large"; from it the following can be extracted: "weekly summary of the clean time; average time for concept location; concept location is becoming faster or slower; recurring exception to the SIP; ..." (IntroSwProcesses p.15). Takeaway: four named derived signals — summary, average, trend, recurring exceptions — the log itself is never the deliverable.

Slides 16–17 — Program size and LOC

Slide 16: program size as "number of lines of source code — LOC, KLOC, MLOC"; "this measure is very inaccurate" because "different programming languages = different size" and "different programming styles = different size" (IntroSwProcesses p.16). Slide 17: "LOC is the most commonly used measure of program size"; "only the one or two most significant digits are meaningful — 900 LOC, 23 KLOC, 3.2 MLOC" (IntroSwProcesses p.17). Takeaway: the *two* stated sources of inaccuracy (languages, styles) plus the significant-digits rule with its three example magnitudes.

Slide 18 — Other measures

"**Function points** — correlate with LOC, harder to compute." "Measures even less accurate than LOC: number of methods in the source code; number of classes; number of files" (IntroSwProcesses p.18). Takeaway: exactly three sub-LOC measures are listed — methods, classes, files.

Slide 19 — Code defects

Defects: "incorrect computations, premature termination." **Defect density:** "good quality software: ~2.0 defects per KLOC; poor quality software: higher density; avionics software — defect density estimated 0.1 defects per KLOC; cutting edge of what can be achieved; NASA Space Shuttle" (IntroSwProcesses p.19). Takeaway: definition + three density anchors (good $\approx$ 2.0, poor = higher, avionics $\approx$ 0.1/Space Shuttle).

Slide 20 — Defect log

The three worked rows detailed in Worked Example D — `Cashier.get()` non-terminating loop (found CL, origin Act, fixed 12/8); the third cashier's missing pop-up (found Base, origin unknown, fixed 11/25); `Price.get()` exception at Price = 0 (found Base, origin Ref); a fourth row marked "..." (IntroSwProcesses p.20). Takeaway: columns are Found(Date/Time/Task) / Location / Description / Origin(Date/Task) / Fixed; entries may legitimately be -- or ?.

Slides 21–22 — Planning and repetitions

Slide 21: "planning is a prediction of the future — there are uncertainties and risks involved"; "data about the past are good predictors about the future"; "recording the past and planning the future are closely related"; the "future cannot be predicted with any level of certainty without knowing the past" (IntroSwProcesses p.21). Slide 22: "easiest things to predict are the repetitions"; "eliminating the risk and uncertainty is one of the main topics of the planning"; "emphasize the repetitive nature of the process — 'repetition is the mother of skill'; "unique and unprecedented tasks are hard to plan" (IntroSwProcesses p.22). Takeaway: planning rests on recorded history; repetition is the lever that makes prediction possible.

Slides 23–25 — Estimation techniques, tasking, and the coupons example

Slide 23: **Analogy** — "estimate the time needed for the phases; find similar phases in the past, use their numbers as the basis"; **Decomposition** — "decompose the change into phases; get the sum for all phases; errors may compensate each other" (IntroSwProcesses p.23). Slide 24: **Tasking** — "changes should be made more alike — they will be more predictable"; "narrow range of size"; "'epics' — divide large changes into smaller ones" (IntroSwProcesses p.24). Slide 25, the tasking example: the epic "Customers can download and then use sales coupons online" decomposes into four subtasks — "build a web site for the store and coupons; support the store manager to create and remove coupons; customer payment involves cashing coupons; database that stores how many coupons were used" (IntroSwProcesses p.25). Takeaway: two estimation techniques, the error-compensation property of decomposition, and the complete four-subtask split of the coupons epic.

Slides 26–27 — Baselines and the release plan

Slide 26: schedule baselines "at regular intervals — every day at the end of the shift; every other day after a change is finished"; "**postponing is not recommended**" (IntroSwProcesses p.26). Slide 27: the release plan "involve[s] business considerations" — "release with a certain new functionality on a certain date; planning makes sure that this promise is realistic" (IntroSwProcesses p.27). Takeaway: baselines are the engineering cadence; the release plan is the business commitment that planning must keep honest.

Slides 28–31 — The release backlog tables

Slide 28 presents the **original** release backlog table (the 0-hours plan; 12 changes; remaining effort 330 = total effort to goal 330) (IntroSwProcesses p.28). Slide 29 shows the table **after 100 hours of work** — four estimates grown (inventory 50, multiple prices 40, cashier sessions 80, multiple line items 70), remaining

340, total-to-goal 440 (IntroSwProcesses p.29). Slide 30 is the **late release** — five snapshots (0/100/285/405/475), further growth (multiple cashiers 55, credit payment 50), remaining reaching 0 at 475 hours with total-to-goal 475 (IntroSwProcesses p.30). Slide 31 is the **incomplete release** — the same table cut at 405 hours, shipping without the last 70 hours of work, i.e. items 11 (credit payment, 50) and 12 (check payment, 20) (IntroSwProcesses p.31). Takeaway: one table, three endings — on paper, late, or incomplete; full numbers in Worked Example E.

Slide-by-Slide Walkthrough — TeamProcesses.pdf (18 slides)

Slides 1–2 — Title and motivation

Slide 1: "Team Iterative Processes (SB5-MAI)," Jan Corfixen Sørensen, University of Southern Denmark. Slide 2: "Most of the software projects require a larger effort than a solo programmer can handle — programmers have to organize themselves into teams" (TeamProcesses p.1–2). Takeaway: capacity, not preference, forces teams; everything after is about the coordination cost that follows.

Slides 3–4 — AIP section divider and characteristics

Slide 3 is the divider "Agile Iterative Processes (AIP)." Slide 4 gives AIP's four characteristics: "agile process for small-to-medium-sized teams; decisions made by consensus; no specializations among the programmers; developers have only the programmer role," presented around the DEVELOP → REACT → MODIFY cycle (TeamProcesses p.3–4). Takeaway: all four characteristics are quotable; the micro-cycle has exactly three verbs.

Slide 5 — Model of AIP

The model: **Users** send **requests** to the **product backlog**; an **iteration backlog** is drawn; **Programmers** perform **parallel software changes**; a **build** is produced; a **daily meeting** coordinates; an **iteration meeting/release** closes the loop; **Product Manager** and **Process Manager** are attached (TeamProcesses p.5). Takeaway: compared with SIP's model, the new elements are *parallelism*, the *two meetings*, and the *two manager roles*.

Slide 6 — Iterations

The **iteration meeting** does two things: "assessing current state of the product" (all stakeholders participate; technical and business point of view) and "planning the next iteration" (the **iteration backlog**, "extracted from product backlog") (TeamProcesses p.6). Takeaway: assessment + planning; the iteration backlog's provenance (extracted from the product backlog) is a frequent fill-in-the-blank.

Slide 7 — Daily meeting

Six listed functions: "daily problems and challenges; a consensus about the progress; daily assignments of change request; clarify the ambiguities; needs for code refactoring; early warning when anything goes wrong" (TeamProcesses p.7). Takeaway: a complete six-item enumeration — exam answers should not stop at "stand-up status."

Slide 8 — Agile Manifesto

"Developed in 2001 (17 original authors); signed by numerous people since." The four value statements: "Individuals and interactions over processes and tools; Working software over comprehensive documentation;

Customer collaboration over contract negotiation; Responding to change over following a plan" (TeamProcesses p.8). Takeaway: year, author count, and all four statements verbatim, left side and right side.

Slides 9–10 — DIP section divider and characteristics

Slide 9 is the divider "Directed Iterative Processes (DIP)." Slide 10: "process runs under direction of managers; several different specialized roles for the programmers; the process scales to large teams and large systems" (TeamProcesses p.9–10). Takeaway: direction + specialization + scale — the three-part DIP definition.

Slide 11 — Model of DIP

The model: **Users** and the **Product manager** feed the **Product backlog**; an **Iteration backlog** is drawn; **Developers** perform **parallel software changes**; **Testers** handle the **Build**; **Process managers** run the **Iteration review/release** (TeamProcesses p.11). Takeaway: in the picture, each box has an owner — that visual assignment of work to roles is the difference from AIP's model.

Slide 12 — The Roles

"Developers produce code; testers verify new baseline; there can be additional specialized roles; specialization increases effectiveness" (TeamProcesses p.12). Takeaway: the open-ended clause ("additional specialized roles") plus the effectiveness claim — specialization is DIP's argument, coordination is its cost.

Slide 13 — Architect

Two responsibilities: "guarantees that developers preserve software architecture constrains [constraints]" and "approves or disapproves commits" (TeamProcesses p.13). Takeaway: the architect both guards design rules and exercises commit power — the embryo of CIP's gate.

Slide 14 — Management

"Product managers make strategic decisions; process managers assign tasks and control the process" (TeamProcesses p.14). Takeaway: the strategic-vs-operational split between the two manager roles, one line each.

Slides 15–16 — CIP section divider and model

Slide 15 is the divider "Centralized Iterative Processes (CIP)." Slide 16 shows the model: **Users** send **requests**; the **product backlog** is fed (Product Manager attached); **Developers** perform **parallel software changes**; the distinctive element — "**permission to commit**" — is granted by **Architects and Code owners**; **Testers** produce the **build**; the **Process Manager** runs the **release** (TeamProcesses p.15–16). Takeaway: the phrase "permission to commit" with its grantors (architects *and* code owners) is CIP's signature; everything else mirrors DIP.

Slide 17 — Code ownership

"Programmers specialize in certain parts of the code; coordination can become a problem" (TeamProcesses p.17). Takeaway: two bullets — the benefit (specialization per code area) and the cost (coordination), both required in an answer.

Slide 18 — Open Source Development

Three characteristics: "Safeguarded; Code ownership; Wide community of developers, variable skills" (TeamProcesses p.18). Takeaway: open source = CIP-style safeguarding + ownership + an open, variable-skill community; the deck closes by placing open source in the same taxonomy as AIP/DIP/CIP.

Slide-by-Slide Walkthrough — ContinuousIntegration.pdf (14 slides)

Slide 1 — Title

"Continuous Integration (SB5-MAI)," Jan Corfixen Sørensen, University of Southern Denmark (ContinuousIntegration p.1).

Slide 2 — Continuous Integration (definition and value)

Five bullets: "Continuous Update for the whole software process; teams integrate their work multiple times per day; each integration is verified by an automated build; significantly reduces integration problems; develop cohesive software more rapidly" — beside the CI cycle image (Commit → Build → Test → Report around source control, returning to Development) (ContinuousIntegration p.2). Takeaway: definition (multiple daily integrations + automated verification) and the two benefits (fewer integration problems, faster cohesive development).

Slide 3 — Five Principles of Continuous Integration

The complete list: "Environments based on stability; Maintain a code repository; Commit frequently and build every commit; Make the build self-testing; Keep the build fast" (ContinuousIntegration p.3). Takeaway: five items, recite in order; slides 4–11 elaborate them one by one.

Slide 4 — Environments based on stability

"Create server environments to model code stability; promote code to stricter environments as quality improves" — the Dev → Test → Stage → Prod promotion picture (ContinuousIntegration p.4). Takeaway: environments form a strictness gradient; promotion is earned by quality.

Slide 5 — Commit Frequently, Build Every Commit

"Change your habits; commit small, functional changes; unit tests!; team owns the code, not the individual" (ContinuousIntegration p.5). Takeaway: the behavioral demands on developers — small functional commits, unit tests, collective ownership (the explicit counterpoint to TeamProcesses' code ownership).

Slide 6 — "The code builds on my box..."

"Source code repository is the source of record; build server settles disputes; only gets code from Repo; build server the final authority on stability/quality" (ContinuousIntegration p.6). Takeaway: the slide title is the anti-pattern; the four bullets are the institutional answer to it.

Slide 7 — Build every commit (why)

"Why compile frequently? Agile principles: **if it hurts, do it more often** — difficult activities can be made more straightforward by doing them more frequently; reduce time between defect introduction and removal;

automate the build" (ContinuousIntegration p.7). Takeaway: two mechanisms (practice effect; shorter defect-to-detection gap) plus the enabling imperative (automation).

Slide 8 — Add testing to build

"Individual programmers < 50% efficient at finding their own bugs; multiple quality methods lead to more defects discovered; use 3 or more methods for > 90% defect removal; most effective methods: design inspections, code inspections, testing" (ContinuousIntegration p.8). Takeaway: three numbers/claims and a three-item method list — all four bullets are exam-grade facts.

Slide 9 — Self Testing Builds

"System tests: end-to-end test; often take minutes to hours to run. **Unit tests:** fast — no database or file system; focused — pinpoint problems; **best method for verifying builds**" (ContinuousIntegration p.9). Takeaway: the contrast table in prose; "no database or file system" is the literal definition of fast here.

Slide 10 — Automated Quality with Continuous Integration

"Static code analysis: looks for common java bugs (Findbugs, PMD); check for code compliance (Checkstyle). **Unit test analysis:** measure coverage (Cobertura); look for hotspots, areas of low testing and high complexity (SONAR)" (ContinuousIntegration p.10). Takeaway: two analysis families, each with two named tools and their exact purposes.

Slide 11 — Build Server Hardware

"Maven and Java = lots of memory; compile and unit test = lots of CPU; static analysis = lots and lots of CPU; **Please, KEEP IT FAST**" (ContinuousIntegration p.11). Takeaway: the resource ledger behind principle 5 — three demand lines and the imperative.

Slide 12 — Test Frameworks

The complete list: "JUnit; NUnit; MSTest; Selenium; FitNesse" (ContinuousIntegration p.12). Takeaway: five frameworks — JUnit (Java unit), NUnit/MSTest (.NET unit), Selenium (browser), FitNesse (acceptance).

Slide 13 — Static Analysis

The complete list: "Checkstyle; CodeScanner; DRY; Crap4j; Findbugs; PMD; Fortify; Sonar; FXCop" (ContinuousIntegration p.13). Takeaway: nine tools — be able to recognize every name as belonging to this category.

Slide 14 — Code Coverage

The complete list: "Emma; Cobertura; Clover; GCC/GCOV" plus the coverage screenshot (green covered lines, red uncovered, "1 of 2 branches missed" on an `addAll(int index, Collection c)` method) (ContinuousIntegration p.14). Takeaway: four coverage tools and what a coverage report visually shows.

Lab Deep Dive — AnalysisLab and CILab

AnalysisLab — Impact Analysis Lab (2 pages)

Introduction (verbatim definition). "Change impact analysis (IA) can be defined as 'identifying the potential consequences of a change, or estimating what needs to be modified to accomplish a change'" (AnalysisLab p.1). This is the lab's quotable definition of IA — note it has *two* clauses (consequences of a change; what must be modified to accomplish it), and exam answers should give both.

Objectives. A single objective: "Apply **static and dynamic analysis** to find the estimated impacted set of classes based on **your Change request**" (AnalysisLab p.1). Two analysis modes, one deliverable (the estimated impact set), anchored to the student's own change request from the course project.

Classwork. "Find The Estimated Impact Set of classes by following the activities illustrated in **Figure 7.9 in [Raj13]**" (AnalysisLab p.1) — i.e. execute the Rajlich Chapter 7 IA activity diagram (the same algorithm as ImpactAnalysis p.12/p.16: BLANK all classes, seed the concept-location class CHANGED, queue BLANK neighbors as NEXT, inspect until no NEXT remains) against the real project codebase.

Portfolio Work. "Use Table 1 to list the **packages** and the **number of classes you visited** after you located the concept. Write short comments explaining **what you have learned about each package** and **how they contribute to your feature**" (AnalysisLab p.1). Table 1, on page 2, is captioned "The list of all the packages visited during impact analysis" and has three columns: **Package name / # of classes / Comments** (AnalysisLab p.2). Two details matter: the count starts *after* concept location (IA is what happens after the concept is located), and the granularity of reporting is the *package* with a per-package class count — the lab aggregates the class-level marking walk up to package level for the portfolio.

What the lab proves you can do. Operationally: build/inspect the interaction structure of a real system (static analysis), run the program to confirm which classes actually participate in the feature (dynamic analysis), execute the Figure 7.9 marking loop, and report the resulting estimated impact set as a package/class table with learned commentary (AnalysisLab p.1–2).

CILab — Continuous Integration Lab (1 page)

Introduction (history paragraph, all three citations). "In software engineering, continuous integration (CI) is the practice of merging all developers' working copies to a shared mainline several times a day [Tho]. Grady Booch first proposed the term CI in his 1991 method although he did not advocate integrating several times a day [Boo]. Extreme programming (XP) adopted the concept of CI and did advocate integrating more than once per day — perhaps as many as tens of times per day [Bec99]" (CILab p.1).

Objectives. Two: "Understand what CI is" and "Setup a simple CI pipeline" (CILab p.1).

Classwork (all five steps). (1) Go to "Building and testing Java with Maven"; (2) add a `*.yaml` file to the repository path `<YOUR_PROJECT>/github/workflows/` "to tell GitHub Actions CI what to do"; (3) configure the `*.yaml` to automatically build the project **for each pull request** (use Maven); (4) to use shared jars from **GitHub Packages**, create a `.maven-settings.xml` file in the project root folder (see "Working with the Apache Maven registry"); (5) configure the `*.yaml` to **execute tests automatically** (CILab p.1).

References. [Bec99] Kent Beck, *Embracing change with extreme programming*, 1999; [Boo] Grady Booch, *Object Oriented Design: With Applications*; [Tho] ThoughtWorks, *Continues Integration* (sic) (CILab p.1).

Toolchain summary. The CI lab's concrete stack is **GitHub Actions** (workflow `*.yaml` under `.github/workflows/`), **Maven** (build tool), **GitHub Packages** (shared jar registry, wired via `.maven-`

`settings.xml`), with automated test execution in the workflow (CILab p.1). The build trigger taught in the lab is *per pull request* — the practical realization of "commit frequently and build every commit" (ContinuousIntegration p.3, p.5).

How the labs map to the decks

Lab element	Deck concept it exercises	Source
AnalysisLab definition of IA	IA "determines the strategy and impact of change"; produces the estimated impact set	AnalysisLab p.1; ImpactAnalysis p.3
"Static and dynamic analysis" objective	Dependency-type edges (static contracts) vs. coordination-type edges (runtime dataflow, <code>b.paint(a.get())</code>)	AnalysisLab p.1; ImpactAnalysis p.5, p.8
Figure 7.9 [Raj13] classwork	The marking algorithm — BLANK/NEXT/CHANGED/UNCHANGED/PROPAGATING/INSPECTED loop	AnalysisLab p.1; ImpactAnalysis p.11–12, p.16
Table 1 (packages / # classes / comments)	The estimated impact set, reported at package granularity	AnalysisLab p.1–2; ImpactAnalysis p.3–4
<code>*.yaml</code> per-pull-request Maven build	Commit frequently and build every commit; automate the build	CILab p.1; ContinuousIntegration p.3, p.5, p.7
Automatic test execution in the workflow	Make the build self-testing; unit tests best for verifying builds	CILab p.1; ContinuousIntegration p.3, p.9
<code>.maven-settings.xml</code> + GitHub Packages	Repository/registry as the source of record; build server only gets code from the repo	CILab p.1; ContinuousIntegration p.6
Booch/XP history paragraph	CI's definition and its "multiple times per day" cadence	CILab p.1; ContinuousIntegration p.2

Compare & Contrast Tables

SIP vs. AIP vs. DIP vs. CIP vs. Open Source

Dimension	SIP	AIP	DIP	CIP	Open Source
Team	single programmer ("Sol")	small-to-medium team	large teams, large systems	large teams (DIP-like)	wide community, variable skills
Decision making	the solo programmer	consensus	direction of managers	direction of managers + gate	safeguarded maintainership
Specialization	none (one person does all)	none among programmers (programmer role only)	several specialized roles	specialized roles incl. code owners	code ownership
Commit control	none needed	none (no gate in the model)	architect approves/disapproves commits	explicit permission to commit from Architects and Code owners	safeguarded

Dimension	SIP	AIP	DIP	CIP	Open Source
Roles in the model	Sol	Programmers, Product Manager, Process Manager	Developers, Testers, Architect, Product manager, Process managers	Developers, Testers, Architects and Code owners, Product Manager, Process Manager	community + owners
Coordination devices	personal logs, baselines	daily meeting + iteration meeting	managers, role boundaries	managers + the commit gate	ownership boundaries
Source	IntroSwProcesses p.7–10	TeamProcesses p.4–7	TeamProcesses p.10–14	TeamProcesses p.16	TeamProcesses p.18

Dependency vs. interaction

	Dependency	Interaction
Definition	one class depends on the other; a contract between them	two classes have something in common (dependency or coordination)
Direction	directed (A → B)	undirected — change propagates from A to B or B to A
Diagram	dashed directed arrows (C → A, C → B)	plain lines, plus extra coordination edges (A—B)
Right tool for	build order, layering, compile-time questions	impact analysis — reachability of change
Source	ImpactAnalysis p.5, p.9	ImpactAnalysis p.5, p.8–9

The three presentations of the IA algorithm

	Marks table (p.11)	IA incl. propagating classes (p.12)	Interactive IA (p.16)
Marks used	BLANK, CHANGED, UNCHANGED, NEXT	+ PROPAGATING; the not-impacted action box reads "Mark class as INSPECTED" under guard [UNCHANGED]	+ INSPECTED as the branch guard itself
Who decides	(definitions only)	undivided activity diagram	Computer does bookkeeping; Programmer decides the new mark
Frontier expansion	n/a	CHANGED and PROPAGATING loop back through "mark all BLANK neighbors as NEXT"	same
Source	ImpactAnalysis p.11	ImpactAnalysis p.12	ImpactAnalysis p.16

Total time vs. clean time

	Total time	Clean time
Includes interruptions?	yes — "total time includes all interruptions"	no — end - start - time of interruptions
Analogy	the full elapsed length of a football game	the 15 minutes of each quarter
Use	scheduling wall-clock	predicting future effort
Worked value (IA row)	31 min (8:52–9:23)	19 min (31 – 12)
Source	IntroSwProcesses p.14	IntroSwProcesses p.13–14

Analogy vs. decomposition (estimation)

	Analogy	Decomposition
Method	find similar phases in the past; use their numbers as the basis	decompose the change into phases; sum all phases
Precondition	a comparable precedent exists	none — works for novel changes
Error behavior	inherits the precedent's representativeness	per-phase errors may compensate each other
Helped by	tasking (changes made more alike)	the phase structure of the change process
Source	IntroSwProcesses p.23	IntroSwProcesses p.23–24

Product backlog vs. iteration backlog vs. release backlog table

	Product backlog	Iteration backlog	Release backlog table
What	all change requests — the vision, bugs, new demands and ideas	the slice extracted from the product backlog for one iteration	per-change remaining-effort estimates snapshotted over hours worked
Produced at	continuously, from users	the iteration meeting	each re-planning point (columns at 0/100/285/405/475 h)
Read for	what the software should become	what the team does next	on-time vs. late vs. incomplete convergence
Source	IntroSwProcesses p.9	IntroSwProcesses p.10; TeamProcesses p.6	IntroSwProcesses p.28–31

Unit tests vs. system tests (in the self-testing build)

	Unit tests	System tests
Speed	fast — no database or file system	often take minutes to hours
Focus	pinpoint problems	end-to-end
Role in CI	best method for verifying builds	too slow for per-commit verification
Source	ContinuousIntegration p.9	ContinuousIntegration p.9

Code ownership vs. "team owns the code"

	Code ownership (CIP / open source)	Collective ownership (CI practice)
Who may change a part	the specializing owner (approves changes in CIP's gate)	anyone — "team owns the code, not the individual"
Benefit	deep familiarity, accountability, staffed safeguard	no gatekeeping; frequent small commits stay frictionless
Cost	coordination can become a problem	relies on automated build/tests to keep quality
Source	TeamProcesses p.16–18	ContinuousIntegration p.5

The three release scenarios side by side

	On-time (original plan)	Late release	Incomplete release
Remaining effort	330 → 0 within 330 h (the plan)	330 → 340 → 180 → 70 → 0	330 → 340 → 180 → 70, then ship
Total effort to goal	stays 330	330 → 440 → 465 → 475	330 → 440 → 465 → 475
Outcome	promised scope on promised effort	full scope, 475 h (≈145 h over)	ships at 405 h without credit payment (50) + check payment (20)
Source	IntroSwProcesses p.28–29	IntroSwProcesses p.30	IntroSwProcesses p.31

CI tooling by category (complete slide lists)

Category	Tools (complete as listed)	Source
Test frameworks	JUnit, NUnit, MSTest, Selenium, FitNesse	ContinuousIntegration p.12
Static analysis	Checkstyle, CodeScanner, DRY, Crap4j, Findbugs, PMD, Fortify, Sonar, FXCop	ContinuousIntegration p.13
Code coverage	Emma, Cobertura, Clover, GCC/GCOV	ContinuousIntegration p.14
Quality analysis roles	Findbugs/PMD = common Java bugs; Checkstyle = code compliance; Cobertura = coverage; SONAR = hotspots (low testing + high complexity)	ContinuousIntegration p.10
Lab pipeline	GitHub Actions (<code>*.yaml</code>), Maven, GitHub Packages (<code>.maven-settings.xml</code>)	CILab p.1

Cross-Lecture and Textbook Connections

Rajlich textbook anchors

- The **ImpactAnalysis** deck carries [Raj13] Chapter 7 attributions on its book-derived slides — the coordination example is annotated as Ch. 7 slide 16, the dependency/interaction diagram contrast as Ch. 7 slide 17, and the marks table as Ch. 7 slide 19 (ImpactAnalysis p.8–9, p.11). The **AnalysisLab** classwork sends students directly to "**Figure 7.9 in [Raj13]**" for the IA activities (AnalysisLab p.1) — so Rajlich Ch. 7 is the textbook home of impact analysis.
- The **IntroSwProcesses** deck's first heading, "**12** Introduction to software processes" (IntroSwProcesses p.1), carries the chapter-12 numbering — read it alongside Rajlich's introduction-to-processes chapter.
- The **TeamProcesses** deck's AIP model carries the [Raj13] Ch. 13 attribution (TeamProcesses p.5) — team iterative processes are Rajlich Ch. 13 material.

How Lecture 3's pieces interlock with the rest of the course

- **Concept Location** → **Impact Analysis**: IA's input is, by definition, Concept Location's output — "classes identified in concept location are the initial impact set" (ImpactAnalysis p.3). Any exam narrative of a change must hand off from CL to IA at exactly that point, as the enactment table does (CL row 3 finds `CashierRecord`; IA row 4 reports "estimated set has 4 classes," IntroSwProcesses p.11).
- **Impact Analysis** → **Prefactoring/Actualization**: the estimated impact set is the worklist of the editing phases; in the enactment, IA's 4-class estimate is followed by Ref "Extract class Session" and Act

"Replace class Session" (IntroSwProcesses p.11; ImpactAnalysis p.3).

- **Baselines → Continuous Integration:** the manual discipline "schedule baselines at regular intervals; postponing is not recommended" (IntroSwProcesses p.26) is what CI mechanizes — "teams integrate their work multiple times per day; each integration is verified by an automated build" (ContinuousIntegration p.2). Baselining is the process-level concept; CI is its automated, team-scale enforcement.
- **Architect's commit approval → CIP gate → code review practice:** the DIP architect "approves or disapproves commits" (TeamProcesses p.13); CIP turns that into an explicit, mandatory "permission to commit" granted by Architects and Code owners (TeamProcesses p.16); open source institutionalizes the same safeguard for a variable-skill community (TeamProcesses p.18).
- **Verification thread:** the cross-cutting VERIFICATION bar (ImpactAnalysis p.3) shows up as the Base step finding 2 regression faults (IntroSwProcesses p.11), the defect log linking found-task to origin-task (IntroSwProcesses p.20), the testers' role of verifying new baselines (TeamProcesses p.12), and CI's self-testing build (ContinuousIntegration p.9) — one idea, four instantiations across the lecture's decks.
- **Measurement thread:** time log → clean time → planning (IntroSwProcesses p.13–14, p.21) parallels CI's automated quality measurement (coverage, hotspots) (ContinuousIntegration p.10) — both are the "performance" form of a process feeding improvement (IntroSwProcesses p.6).

Self-Test Question Bank

Answers cite the slide that grounds them. Cover the answer column and quiz yourself.

Impact Analysis questions

1. **Q:** What does Impact Analysis determine, and what are its input and output sets? **A:** It "determines the strategy and impact of change"; input = the initial impact set (classes identified in concept location), output = the estimated impact set, built by analyzing class dependencies and adding impacted classes (ImpactAnalysis p.3).
2. **Q:** Give the lab's two-clause definition of change impact analysis. **A:** "Identifying the potential consequences of a change, or estimating what needs to be modified to accomplish a change" (AnalysisLab p.1).
3. **Q:** When do two classes interact, and what are the two flavors? **A:** When they have something in common — either one depends on the other (a contract between them), or they coordinate (share the same coding, schedule, etc.) (ImpactAnalysis p.5).
4. **Q:** In which direction(s) do interactions propagate change? **A:** Both — from A to B or from B to A (ImpactAnalysis p.5).
5. **Q:** Define the class interaction graph and the neighborhood of a class. **A:** $G = (X, I)$, X the set of classes, I the set of interactions; $N(A) = \{B \mid (A, B) \in I\}$ (ImpactAnalysis p.6).
6. **Q:** In the Store model, what is $N(\text{Item})$? **A:** {Inventory, Price} — the classes directly connected to Item (ImpactAnalysis p.7).
7. **Q:** Write the one-line code example of coordination and explain it. **A:** `b.paint(a.get());` inside `C.foo()` — a dataflow between `a` and `b`, so A and B interact although neither references the other; the link lives in C's body (ImpactAnalysis p.8).
8. **Q:** How do the dependency diagram and the interaction diagram of the same classes differ? **A:** The dependency diagram has directed arrows $C \rightarrow A$ and $C \rightarrow B$; the interaction diagram has undirected lines and an additional $A-B$ coordination edge (ImpactAnalysis p.9).
9. **Q:** Retell the mailman analogy and name what the mailman models. **A:** John (changed situation: wants his loan back) writes to Paul; the mailman carries the letter; Paul must take a part-time job — a big change

propagated from John through the mailman to Paul. The mailman models a propagating class: unchanged itself, yet it relays change (ImpactAnalysis p.10).

10. **Q:** Define all four marks of the p.11 table. **A:** Blank — never inspected, not scheduled; Changed — inspected and found impacted; Unchanged — inspected and found not impacted; Next — scheduled for inspection (ImpactAnalysis p.11).
11. **Q:** Which verdicts cause the algorithm to mark more neighbors NEXT? **A:** CHANGED and PROPAGATING — both loop back through "mark all BLANK neighbors as NEXT"; the UNCHANGED/INSPECTED branch does not expand the frontier (ImpactAnalysis p.12, p.16).
12. **Q:** What is split between the swimlanes in Interactive IA? **A:** The Computer creates the diagram and manages all marking/queueing; the Programmer selects a NEXT class and decides its new mark ([INSPECTED]/[PROPAGATING]/[CHANGED]) (ImpactAnalysis p.16).
13. **Q:** Name the two criteria for weighing change alternatives and the conflict between them. **A:** Required effort of the change vs. clarity of the resulting code; they often contradict — easier to adjust the UI, better to keep all temperature calculations in one place — a short-term vs. long-term goals conflict (ImpactAnalysis p.14–15).

Software process questions

14. **Q:** Why is the study of software processes called the core of software engineering? **A:** Successful past projects yield processes that worked well — a prescription for future projects; unsuccessful ones reveal the problems that led to failure; good process → good product (IntroSwProcesses p.1).
15. **Q:** Along which two dimensions do processes vary? **A:** Team (organization, collaboration, skills) and System (technology, domain, size, expected quality) (IntroSwProcesses p.2).
16. **Q:** Reproduce the granularity table. **A:** lifecycle — staged, waterfall; stage — evolution, servicing; process — SIP, AIP, DIP; task — software change, acceptance testing; subtask/phase — concept location, actualization; step/action — inspection of a class (IntroSwProcesses p.4).
17. **Q:** Define the four forms of process. **A:** Model = prescription/blueprint; enactment = the actual process with inevitable deviations; performance = measures an observer collects (time, cost, quality); plan = expected future performance, stakeholders' decisions among enactment alternatives (IntroSwProcesses p.5–6).
18. **Q:** What is SIP and why does even a solo programmer need it? **A:** A single programmer repeats software changes, adding functionality one step at a time; even solo programmers must meet obligations — fulfill promises, pay bills, plan the future, manage their own resources — and SIP is the process that allows that (IntroSwProcesses p.7–8).
19. **Q:** List SIP's workproducts. **A:** Product backlog (change requests; the vision for the software's future; bugs; new demands and ideas), software code, software documentation, etc. (IntroSwProcesses p.9).
20. **Q:** In the SIP enactment example, what did IA report and what happened at the baseline? **A:** "Estimated set has 4 classes"; at Base, 2 regression faults were added to the backlog (IntroSwProcesses p.11).
21. **Q:** Give the clean-time formula and the football analogy. **A:** Clean time = end – start – time of interruptions; 15 minutes is the clean time of each quarter while total time includes all interruptions (IntroSwProcesses p.14).
22. **Q:** Name four things extractable from the (large) time log. **A:** Weekly summary of clean time; average time for concept location; whether concept location is becoming faster or slower; recurring exceptions to SIP (IntroSwProcesses p.15).
23. **Q:** Why is LOC inaccurate, and how many digits of it are meaningful? **A:** Different languages and different styles give different sizes; only the one or two most significant digits are meaningful (900 LOC, 23 KLOC, 3.2 MLOC) (IntroSwProcesses p.16–17).

24. **Q:** Which size measures are even less accurate than LOC, and which correlates with LOC? **A:** Number of methods, number of classes, number of files are less accurate; function points correlate with LOC but are harder to compute (IntroSwProcesses p.18).
25. **Q:** Define a defect and give the three density anchors. **A:** Incorrect computations or premature termination; good software ~2.0 defects/KLOC, poor software higher, avionics ~0.1 defects/KLOC (cutting edge; NASA Space Shuttle) (IntroSwProcesses p.19).
26. **Q:** What columns does the defect log have? **A:** Found (Date, Time, Task), Location, Description, Origin (Date, Task), Fixed (IntroSwProcesses p.20).
27. **Q:** Why can't the future be planned without the past? **A:** Planning is a prediction of the future with uncertainties and risks; data about the past are good predictors; the future cannot be predicted with any certainty without knowing the past (IntroSwProcesses p.21).
28. **Q:** What are the easiest things to predict, and what does tasking do about it? **A:** Repetitions ("repetition is the mother of skill"); tasking makes changes more alike — narrow range of size, dividing large changes ("epics") into smaller ones — so they become more predictable (IntroSwProcesses p.22, p.24).
29. **Q:** Decompose the sales-coupons epic as the slide does. **A:** Build a web site for the store and coupons; support the store manager to create and remove coupons; customer payment involves cashing coupons; a database storing how many coupons were used (IntroSwProcesses p.25).
30. **Q:** State the baseline scheduling rule. **A:** Schedule at regular intervals — every day at the end of the shift, or every other day after a change is finished; postponing is not recommended (IntroSwProcesses p.26).
31. **Q:** After 100 hours, remaining effort is 340. What is the total effort needed to reach the goal, and why? **A:** 440 — hours already worked (100) plus remaining effort (340); four estimates grew (inventory 50, multiple prices 40, cashier sessions 80, multiple line items 70) (IntroSwProcesses p.29).
32. **Q:** How do you recognize an incomplete release in the backlog table? **A:** The table stops with remaining effort > 0 — at 405 hours, 70 hours of work (credit payment 50 + check payment 20) are cut and the release ships without them (IntroSwProcesses p.31).

Team process questions

33. **Q:** Why do programmers organize into teams? **A:** Most software projects require a larger effort than a solo programmer can handle (TeamProcesses p.2).
34. **Q:** Give AIP's four characteristics and its micro-cycle. **A:** Agile process for small-to-medium teams; decisions by consensus; no specializations among programmers; developers have only the programmer role — cycling DEVELOP → REACT → MODIFY (TeamProcesses p.4).
35. **Q:** What two jobs does the iteration meeting do? **A:** Assess the current state of the product (all stakeholders; technical and business viewpoints) and plan the next iteration (the iteration backlog, extracted from the product backlog) (TeamProcesses p.6).
36. **Q:** List the six functions of the daily meeting. **A:** Daily problems and challenges; consensus about progress; daily assignment of change requests; clarify ambiguities; needs for code refactoring; early warning when anything goes wrong (TeamProcesses p.7).
37. **Q:** State the Agile Manifesto's metadata and four values. **A:** Developed 2001 by 17 original authors, signed by numerous people since; individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; responding to change over following a plan (TeamProcesses p.8).
38. **Q:** Define DIP in three clauses. **A:** Runs under the direction of managers; several specialized roles for the programmers; scales to large teams and large systems (TeamProcesses p.10).
39. **Q:** What does the architect do? **A:** Guarantees developers preserve software architecture constraints; approves or disapproves commits (TeamProcesses p.13).

40. **Q:** Distinguish product managers from process managers. **A:** Product managers make strategic decisions; process managers assign tasks and control the process (TeamProcesses p.14).
41. **Q:** What distinguishes CIP's model, and who staffs it? **A:** The "permission to commit" required before developers' parallel changes enter the codebase — granted by Architects and Code owners (TeamProcesses p.16).
42. **Q:** Code ownership: benefit and risk? **A:** Programmers specialize in certain parts of the code (deep familiarity); coordination can become a problem (TeamProcesses p.17).
43. **Q:** Characterize open source development per the slide. **A:** Safeguarded; code ownership; wide community of developers of variable skills (TeamProcesses p.18).

Continuous Integration and lab questions

44. **Q:** Define CI per the deck and per the lab. **A:** Deck: teams integrate their work multiple times per day, each integration verified by an automated build — significantly reduces integration problems and develops cohesive software more rapidly (ContinuousIntegration p.2). Lab: the practice of merging all developers' working copies to a shared mainline several times a day [Tho] (CILab p.1).
45. **Q:** Recite the five principles of CI. **A:** Environments based on stability; maintain a code repository; commit frequently and build every commit; make the build self-testing; keep the build fast (ContinuousIntegration p.3).
46. **Q:** What does "environments based on stability" prescribe? **A:** Create server environments that model code stability and promote code to stricter environments as quality improves (ContinuousIntegration p.4).
47. **Q:** What habit changes does "commit frequently" demand? **A:** Commit small, functional changes; write unit tests; the team owns the code, not the individual (ContinuousIntegration p.5).
48. **Q:** How does CI answer "the code builds on my box"? **A:** The source-code repository is the source of record; the build server settles disputes, only gets code from the repo, and is the final authority on stability/quality (ContinuousIntegration p.6).
49. **Q:** Justify building every commit. **A:** "If it hurts, do it more often" — difficult activities become more straightforward done more frequently; it reduces the time between defect introduction and removal; therefore automate the build (ContinuousIntegration p.7).
50. **Q:** Give the defect-detection statistics behind self-testing builds. **A:** Individual programmers are < 50% efficient at finding their own bugs; multiple quality methods discover more defects — use 3 or more for > 90% defect removal; most effective: design inspections, code inspections, testing (ContinuousIntegration p.8).
51. **Q:** Why are unit tests the best method for verifying builds? **A:** They are fast (no database or file system) and focused (pinpoint problems), unlike end-to-end system tests that often take minutes to hours (ContinuousIntegration p.9).
52. **Q:** Which tool does what in automated quality analysis? **A:** Findbugs and PMD look for common Java bugs; Checkstyle checks code compliance; Cobertura measures coverage; SONAR finds hotspots of low testing and high complexity (ContinuousIntegration p.10).
53. **Q:** What does the build server need, and why the warning? **A:** Maven and Java need lots of memory; compile and unit tests lots of CPU; static analysis lots and lots of CPU — "Please, KEEP IT FAST" because slow builds break the per-commit feedback loop (ContinuousIntegration p.11).
54. **Q:** Sort into categories: Selenium, Crap4j, Clover, FitNesse, FXCop, Emma. **A:** Test frameworks: Selenium, FitNesse; static analysis: Crap4j, FXCop; coverage: Clover, Emma (ContinuousIntegration p.12–14).
55. **Q:** Who coined CI and who advocated integrating tens of times per day? **A:** Grady Booch first proposed the term in his 1991 method (without advocating several integrations a day); Extreme Programming adopted CI and advocated more than once per day — perhaps tens of times (CILab p.1).

56. **Q:** List the five CILab classwork steps. **A:** (1) Follow "Building and testing Java with Maven"; (2) add a `*.yml` under `<YOUR_PROJECT>/github/workflows/`; (3) configure it to build each pull request with Maven; (4) create `.maven-settings.xml` in the project root for shared jars from GitHub Packages; (5) configure the `*.yml` to execute tests automatically (CILab p.1).
57. **Q:** What are the AnalysisLab's deliverables? **A:** The estimated impact set found by following Figure 7.9 in [Raj13], and Table 1 listing every package visited during impact analysis with the number of classes and comments on what was learned about each package and how it contributes to the feature (AnalysisLab p.1–2).